



हां जी तो करें स्टार्ट। वेलकम बै...
TXT 129.25KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Mediator Design Pattern - Complete Notes

1. Introduction & Purpose

The **Mediator Design Pattern** is a behavioral design pattern that:

- Defines an object that encapsulates how a set of objects interact
- Promotes **loose coupling** by preventing objects from referring to each other explicitly
- Allows you to vary their interaction independently

The Problem It Solves

Without Mediator Pattern:

Imagine you have objects OB1 , OB2 , OB3 , OB4 that need to communicate with each other.

text

OB1 ↔ OB2

OB1 ↔ OB3

OB1 ↔ OB4

OB2 ↔ OB3

OB2 ↔ OB4

OB3 ↔ OB4

Problems:

1. **Tight Coupling:** Each object must know about all other objects
2. **$N \times (N-1)$ Connections:** For N objects, each needs N-1 references
3. **Memory Overhead:** Each object maintains a list of all other objects
4. **Hard to Maintain:** Adding a new object requires modifying all existing objects

5. Violates Open-Closed Principle: To add features (like mute), you must modify every class

Real-world Example (Chat Room without Mediator):

java

// BAD DESIGN - Without Mediator

```
class User {
    private String name;
    private List<User> peers = new ArrayList<>();    // List of all other users
    private List<String> mutedUsers = new ArrayList<>(); // List of muted users

    public User(String name) {
        this.name = name;
    }

    public void addPeer(User user) {
        peers.add(user);
    }

    public void mute(User user) {
        mutedUsers.add(user.getName());
    }

    public boolean isMuted(String userName) {
        return mutedUsers.contains(userName);
    }

    public void send(String message) {
        System.out.println(name + " is broadcasting: " + message);
        for (User peer : peers) {
            if (!peer.isMuted(this.name)) {
                peer.receive(this.name, message);
            }
        }
    }

    public void sendTo(User target, String message) {
        System.out.println(name + " is sending privately to " + target.getName());
        if (!target.isMuted(this.name)) {
            target.receive(this.name, message);
        }
    }

    public void receive(String from, String message) {
        System.out.println(name + " got from " + from + ": " + message);
    }
}
```

```

    public String getName() { return name; }
}

// Main - Notice how many connections we need to make!
public class Main {
    public static void main(String[] args) {
        User rohan = new User("Rohan");
        User neha = new User("Neha");
        User mohan = new User("Mohan");

        // N*(N-1) connections - 6 connections for 3 users
        rohan.addPeer(neha);
        neha.addPeer(rohan);
        rohan.addPeer(mohan);
        mohan.addPeer(rohan);
        neha.addPeer(mohan);
        mohan.addPeer(neha);

        rohan.mute(mohan);

        rohan.send("Hello everyone");
        mohan.sendTo(neha, "Hey Neha");
    }
}

```

Output:

text

```

Rohan is broadcasting: Hello everyone
Neha got from Rohan: Hello everyone
Mohan got from Rohan: Hello everyone
Mohan is sending privately to Neha
Neha got from Mohan: Hey Neha

```

2. The Mediator Solution

How It Works

Instead of objects communicating directly, they communicate through a **Mediator**:

text

OB1 → Mediator → OB2

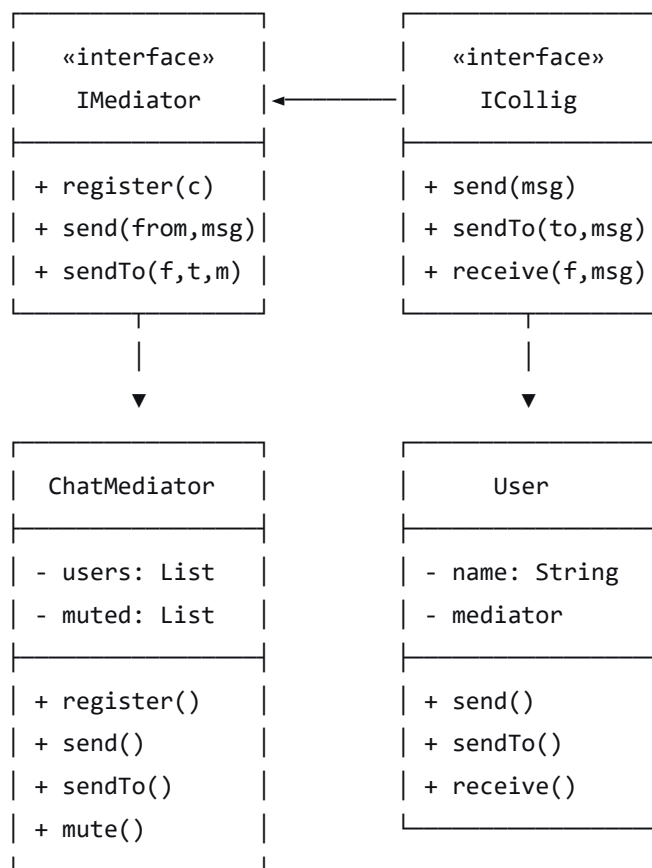
OB3 → Mediator → OB4

Key Points:

- Each object only knows about the Mediator
- The Mediator knows about all objects
- Mediator handles all communication routing
- Objects don't hold references to each other

UML Diagram

text



Components

Component

Role

IMediator

Abstract class/interface defining communication methods

Component	Role
IColleague	Abstract class for objects that communicate via mediator
ConcreteMediator	Implements mediator, maintains list of colleagues
ConcreteColleague	Implements colleague, delegates communication to mediator

3. Java Implementation

IMediator.java (Abstract Mediator)

java

```
// Forward declaration
public interface IMediator {
    void registerColleague(IColleague colleague);
    void send(String from, String message);
    void sendTo(String from, String to, String message);
    void mute(String who, String whom);
}
```

IColleague.java (Abstract Colleague)

java

```
public abstract class IColleague {
    protected IMediator mediator;

    public IColleague(IMediator mediator) {
        this.mediator = mediator;
        mediator.registerColleague(this);
    }

    public abstract String getName();
    public abstract void send(String message);
    public abstract void sendTo(String to, String message);
    public abstract void receive(String from, String message);
}
```

ChatMediator.java (Concrete Mediator)

```

java

import java.util.*;

public class ChatMediator implements IMediator {
    private List<IColleague> colleagues = new ArrayList<>();
    private List<Pair<String, String>> muted = new ArrayList<>();

    // Simple Pair class
    static class Pair<K, V> {
        K first;
        V second;
        Pair(K first, V second) {
            this.first = first;
            this.second = second;
        }
    }

    @Override
    public void registerColleague(IColleague colleague) {
        colleagues.add(colleague);
    }

    @Override
    public void mute(String who, String whom) {
        muted.add(new Pair<>(who, whom));
    }

    @Override
    public void send(String from, String message) {
        System.out.println("\n" + from + " is broadcasting: " + message);

        for (IColleague c : colleagues) {
            // Don't send to self
            if (c.getName().equals(from)) {
                continue;
            }

            boolean isMuted = false;
            for (Pair<String, String> p : muted) {
                // If 'from' is in anyone's muted List (as the muted person)
                // OR 'c' has muted 'from' (c is the muter, from is muted)
                if (p.second.equals(from) || p.first.equals(c.getName()) && p.second.equals(
ls(from)) {
                    isMuted = true;
                    break;

```

```

    }
}

    if (!isMuted) {
        c.receive(from, message);
    }
}

}

@Override
public void sendTo(String from, String to, String message) {
    System.out.println("\n" + from + " is sending privately to " + to);

    for (IColleague c : colleagues) {
        if (c.getName().equals(to)) {
            boolean isMuted = false;
            for (Pair<String, String> p : muted) {
                // Check if 'to' has muted 'from'
                if (p.first.equals(to) && p.second.equals(from)) {
                    isMuted = true;
                    break;
                }
            }

            if (!isMuted) {
                c.receive(from, message);
            } else {
                System.out.println("Message is muted. Cannot deliver.");
            }
            return;
        }
    }
}
}
}

```

User.java (Concrete Colleague)

```

java

public class User extends IColleague {
    private String name;

    public User(String name, IMediator mediator) {
        super(mediator);
        this.name = name;
    }
}

```

```

@Override
public String getName() {
    return name;
}

@Override
public void send(String message) {
    mediator.send(this.name, message);
}

@Override
public void sendTo(String to, String message) {
    mediator.sendTo(this.name, to, message);
}

@Override
public void receive(String from, String message) {
    System.out.println(name + " got from " + from + ": " + message);
}
}

```

Main.java

java

```

public class Main {
    public static void main(String[] args) {
        // Create mediator
        IMediator chatRoom = new ChatMediator();

        // Create users - they auto-register with mediator
        User rohan = new User("Rohan", chatRoom);
        User neha = new User("Neha", chatRoom);
        User mohan = new User("Mohan", chatRoom);

        // No connections needed between users!
        // Rohan mutes Mohan
        chatRoom.mute("Rohan", "Mohan");

        // Rohan broadcasts
        rohan.send("Hello everyone");

        // Mohan sends private message to Neha
        mohan.sendTo("Neha", "Hey Neha");
    }
}

```

Output:

text

Rohan is broadcasting: Hello everyone

Neha got from Rohan: Hello everyone

Mohan got from Rohan: Hello everyone

Mohan is sending privately to Neha

Neha got from Mohan: Hey Neha

4. Python Implementation

python

```

from abc import ABC, abstractmethod
from typing import List, Tuple

# Abstract Mediator
class IMediator(ABC):
    @abstractmethod
    def register_colleague(self, colleague):
        pass

    @abstractmethod
    def send(self, from_user: str, message: str):
        pass

    @abstractmethod
    def send_to(self, from_user: str, to_user: str, message: str):
        pass

    @abstractmethod
    def mute(self, who: str, whom: str):
        pass

# Abstract Colleague
class IColleague(ABC):
    def __init__(self, mediator: IMediator):
        self.mediator = mediator
        mediator.register_colleague(self)

    @abstractmethod
    def get_name(self) -> str:

```

```

    pass

    @abstractmethod
    def send(self, message: str):
        pass

    @abstractmethod
    def send_to(self, to: str, message: str):
        pass

    @abstractmethod
    def receive(self, from_user: str, message: str):
        pass

# Concrete Mediator
class ChatMediator(IMediator):
    def __init__(self):
        self.colleagues: List[IColleague] = []
        self.muted: List[Tuple[str, str]] = [] # (muter, muted)

    def register_colleague(self, colleague: IColleague):
        self.colleagues.append(colleague)

    def mute(self, who: str, whom: str):
        self.muted.append((who, whom))

    def send(self, from_user: str, message: str):
        print(f"\n{from_user} is broadcasting: {message}")

        for c in self.colleagues:
            # Don't send to self
            if c.get_name() == from_user:
                continue

            is_muted = False
            for (muter, muted_person) in self.muted:
                if muted_person == from_user:
                    is_muted = True
                    break
                if muter == c.get_name() and muted_person == from_user:
                    is_muted = True
                    break

            if not is_muted:
                c.receive(from_user, message)

    def send_to(self, from_user: str, to_user: str, message: str):

```

```

print(f"\n{from_user} is sending privately to {to_user}")

for c in self.colleagues:
    if c.get_name() == to_user:
        is_muted = False
        for (muter, muted_person) in self.muted:
            if muter == to_user and muted_person == from_user:
                is_muted = True
                break

        if not is_muted:
            c.receive(from_user, message)
        else:
            print("Message is muted. Cannot deliver.")
    return

# Concrete Colleague
class User(IColleague):
    def __init__(self, name: str, mediator: IMediator):
        super().__init__(mediator)
        self.name = name

    def get_name(self) -> str:
        return self.name

    def send(self, message: str):
        self.mediator.send(self.name, message)

    def send_to(self, to: str, message: str):
        self.mediator.send_to(self.name, to, message)

    def receive(self, from_user: str, message: str):
        print(f"{self.name} got from {from_user}: {message}")

# Main
if __name__ == "__main__":
    chat_room = ChatMediator()

    rohan = User("Rohan", chat_room)
    neha = User("Neha", chat_room)
    mohan = User("Mohan", chat_room)

    # Rohan mutes Mohan
    chat_room.mute("Rohan", "Mohan")

    # Rohan broadcasts

```

```
rohan.send("Hello everyone")

# Mohan sends private message to Neha
mohan.send_to("Neha", "Hey Neha")
```

5. Mediator vs Observer Pattern

Both patterns look similar in UML but have **fundamentally different intents**:

Aspect	Observer Pattern	Mediator Pattern
Intent	Notify observers when state changes	Enable communication between colleagues
Focus	One-to-many dependency	Many-to-many communication
Communication	One-way (Observable → Observers)	Two-way (Colleagues ↔ Mediator ↔ Colleagues)
Purpose	State change notification	Decouple objects from each other
Example	Notification server, Event listeners	Chat room, Air traffic control
Key Method	notify() , update()	send() , receive()

Key Insight:

- **Observer:** "When my state changes, tell everyone who's listening"
- **Mediator:** "Don't talk to each other directly, talk through me"

6. Real-World Use Cases

1. **Chat Rooms:** Multiple users communicating through a central room
2. **Air Traffic Control:** Planes don't communicate directly; control tower mediates
3. **GUI Frameworks:** Dialog boxes coordinate between widgets
4. **Online Gaming:** Players in a match communicate through game server

5. **Stock Exchange:** Buyers and sellers trade through exchange (mediator)
6. **Microservices API Gateway:** Services communicate through gateway

7. Advantages & Disadvantages

Advantages

- ✓ **Loose Coupling:** Objects don't need to know about each other
- ✓ **Centralized Control:** Communication logic in one place
- ✓ **Easy to Maintain:** Add/remove colleagues without changing others
- ✓ **Open-Closed Principle:** Add new features (mute, etc.) in mediator only
- ✓ **Reduces Subclassing:** No need to create communication subclasses

Disadvantages

- ✗ **Single Point of Failure:** If mediator fails, everything fails
- ✗ **Can Become God Object:** Mediator can become too complex
- ✗ **Performance:** Adds indirection layer

8. Key Takeaways

1. **Mediator** defines an object that encapsulates how objects interact
2. It promotes **loose coupling** by preventing direct references
3. **Colleagues** only know about the mediator, not each other
4. **Mediator** knows about all colleagues and routes communication
5. Different from **Observer** in intent — Mediator enables two-way communication; Observer is one-way notification
6. Best used when you have many-to-many object relationships that would otherwise be tightly coupled

9. Interview Tip

When asked about design patterns, explain:

- **What problem it solves** (tight coupling, N^2 connections)
- **How it solves it** (introduces mediator, centralizes communication)
- **When to use it** (chat rooms, air traffic control, GUI dialogs)
- **Difference from similar patterns** (Observer vs Mediator)

The key differentiator: **Mediator focuses on decoupling objects that need to communicate; Observer focuses on state change notification.**



हां जी तो करें स्टार्ट। वेलकम बै...
TXT 85.03KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Prototype Design Pattern - Complete Notes

1. Introduction & Purpose

The **Prototype Design Pattern** is a creational design pattern that:

- Allows you to create new objects by **copying/cloning** an existing object (called a prototype)
- Avoids the expensive cost of creating objects from scratch
- Provides a way to create objects without coupling to their specific classes

Simple Definition

"It lets you create new objects by copying or cloning another instance."

2. The Problem It Solves

Core Problem: Expensive Object Creation

Consider an object whose creation is **expensive**:

text

Object creation cost can include:

- Database connection establishment
- Complex calculations
- Loading files from disk
- Network requests
- Heavy initialization logic

If you need **100 such objects**, calling `new A()` 100 times means:

- 100 database connections
- 100 complex calculations
- 100x the time and memory

Real-World Example: NPCs in a Game

NPC (Non-Player Character) = Computer-generated characters in games

- Examples: pedestrians in GTA, enemies, background characters
- They look almost identical with very few differences
- Creating one NPC can be expensive (loading images, DB queries)

Without Prototype:

```
java
```

```
NPC alien1 = new NPC("Alien", 30, 5, 2);    // Expensive! (5 sec)
NPC alien2 = new NPC("Alien", 30, 5, 2);    // Expensive again!
NPC alien3 = new NPC("Alien", 30, 5, 2);    // Expensive again!
// ... 100 times = 500 seconds wasted
```

With Prototype:

```
java
```

```
NPC alien1 = new NPC("Alien", 30, 5, 2);    // Expensive (once)
NPC alien2 = alien1.clone();                // Fast! Copy
NPC alien3 = alien1.clone();                // Fast! Copy
// ... 100 times = 5 seconds + tiny copy overhead
```

3. Understanding Copy Constructors

The Prototype pattern uses **copy constructors** as its foundation.

Three Types of Constructors

Constructor Type	Description	Example
Default	No arguments	A() {}
Parameterized	Takes arguments	A(int a, int b) {}
Copy	Takes another instance of same class	A(const A& other) {}

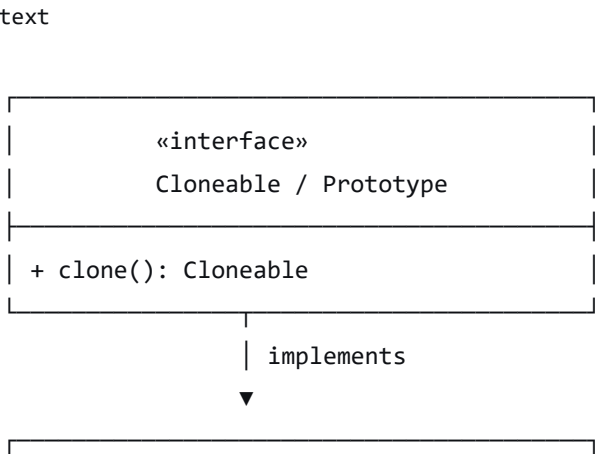
Copy Constructor in Action

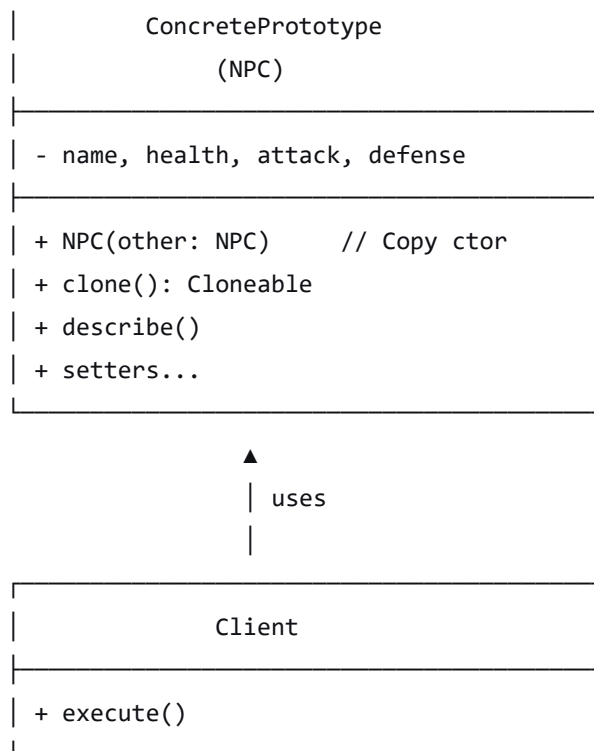
```
cpp
class A {
    int x, y;
public:
    A(int x, int y) : x(x), y(y) {} // Parameterized

    A(const A& other) {                // Copy Constructor
        this->x = other.x;
        this->y = other.y;
    }
};
```

4. Solution: The Prototype Pattern

How It Works





The Cloneable Marker Interface

java

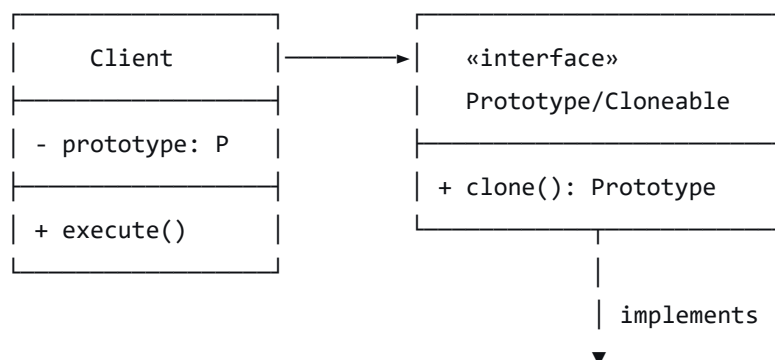
```
// Java has a built-in Cloneable marker interface
public interface Cloneable { } // No methods! Just a marker
```

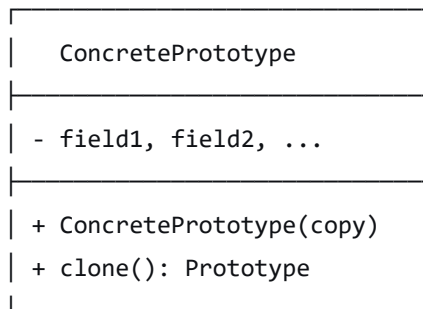
Purpose: To mark classes as "cloneable" — indicates to clients that the object supports cloning.

Similar in concept to C++'s **pure abstract class**.

5. UML Class Diagram

text





Client's execute() method:

text

```
execute():
    clone = prototype.clone()
    return clone
```

6. Java Implementation

Complete Example: NPC Game Characters

Step 1: Cloneable Interface (Marker)

```
java

// Marker interface - no methods, just marks classes as cloneable
public interface Cloneable {
    Cloneable clone();
}
```

Step 2: NPC Class (Concrete Prototype)

```
java

public class NPC implements Cloneable {
    private String name;
    private int health;
    private int attack;
    private int defense;

    // EXPENSIVE Constructor - simulates DB calls, complex calculations
    public NPC(String name, int health, int attack, int defense) {
        System.out.println("Creating NPC: " + name + " (EXPENSIVE OPERATION)");
    }
}
```

```

    // Simulate expensive operations
    try {
        // Fake DB connection
        Thread.sleep(1000); // Simulating 1 second delay
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    // Complex calculations
    // ... (pretend there's heavy work here)

    this.name = name;
    this.health = health;
    this.attack = attack;
    this.defense = defense;
}

// COPY CONSTRUCTOR - This is the KEY to Prototype pattern
public NPC(NPC other) {
    // Copy all values directly - no expensive operations!
    this.name = other.name;
    this.health = other.health;
    this.attack = other.attack;
    this.defense = other.defense;
}

// Clone method - uses copy constructor
@Override
public Cloneable clone() {
    System.out.println("Cloning NPC: " + this.name);
    return new NPC(this); // Calls copy constructor
}

// Describe - just prints current state
public void describe() {
    System.out.println("NPC [name=" + name + ", health=" + health +
        ", attack=" + attack + ", defense=" + defense + "]");
}

// Setters for tweaking cloned objects
public void setName(String name) { this.name = name; }
public void setHealth(int health) { this.health = health; }
public void setAttack(int attack) { this.attack = attack; }
public void setDefense(int defense) { this.defense = defense; }

// Getters
public String getName() { return name; }

```

```
public int getHealth() { return health; }  
public int getAttack() { return attack; }  
public int getDefense() { return defense; }  
}
```

Step 3: Main Class

java

```
public class Main {  
    public static void main(String[] args) {  
        // Step 1: Create the initial prototype (EXPENSIVE - one time)  
        System.out.println("=== Creating Original NPC ===");  
        NPC alien = new NPC("Alien", 30, 5, 2);  
        alien.describe();  
  
        // Step 2: Clone it! (FAST)  
        System.out.println("\n=== Cloning NPC (No expensive operation) ===");  
        NPC alienCopy = (NPC) alien.clone();  
        alienCopy.describe();  
  
        // Step 3: Clone again and customize  
        System.out.println("\n=== Cloning and Customizing ===");  
        NPC powerfulAlien = (NPC) alien.clone();  
        powerfulAlien.setName("Powerful Alien");  
        powerfulAlien.setHealth(50);  
        powerfulAlien.setDefense(5);  
        powerfulAlien.describe();  
  
        // Original is UNCHANGED  
        System.out.println("\n=== Original is unchanged ===");  
        alien.describe();  
    }  
}
```

Expected Output

text

```
=== Creating Original NPC ===  
Creating NPC: Alien (EXPENSIVE OPERATION)  
NPC [name=Alien, health=30, attack=5, defense=2]  
  
=== Cloning NPC (No expensive operation) ===  
Cloning NPC: Alien  
NPC [name=Alien, health=30, attack=5, defense=2]
```

=== Cloning and Customizing ===

Cloning NPC: Alien

NPC [name=Powerful Alien, health=50, attack=5, defense=5]

=== Original is unchanged ===

NPC [name=Alien, health=30, attack=5, defense=2]

Java's Built-in Approach (Bonus)

Java has a built-in `Cloneable` interface and `Object.clone()` method:

java

```
public class NPC implements Cloneable {
    private String name;
    private int health;

    // ... constructor and fields ...

    @Override
    public NPC clone() {
        try {
            return (NPC) super.clone(); // Shallow copy
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Note: Java's `super.clone()` creates a **shallow copy** by default.

7. Python Implementation

Python has built-in support through `copy` module.

python

```
import copy
import time
from abc import ABC, abstractmethod
from typing import Optional

# Marker interface (Python uses ABC for abstractness)
```

```

class Prototype(ABC):
    @abstractmethod
    def clone(self) -> 'Prototype':
        pass

# Concrete Prototype
class NPC(Prototype):
    def __init__(self, name: str, health: int, attack: int, defense: int):
        print(f"Creating NPC: {name} (EXPENSIVE OPERATION)")

        # Simulate expensive operations
        time.sleep(1) # Simulating 1 second delay

        # Complex calculations would go here
        # DB connection, file loading, etc.

        self.name = name
        self.health = health
        self.attack = attack
        self.defense = defense

# Optional: Copy constructor via classmethod
@classmethod
def from_prototype(cls, other: 'NPC') -> 'NPC':
    """Copy constructor equivalent"""
    instance = cls.__new__(cls) # Bypass __init__ to avoid expensive operations
    instance.name = other.name
    instance.health = other.health
    instance.attack = other.attack
    instance.defense = other.defense
    return instance

# Clone method
def clone(self) -> 'NPC':
    print(f"Cloning NPC: {self.name}")
    return NPC.from_prototype(self)

def describe(self):
    print(f"NPC [name={self.name}, health={self.health}, "
          f"attack={self.attack}, defense={self.defense}]")

# Setters
def set_name(self, name: str):
    self.name = name

def set_health(self, health: int):
    self.health = health

```

```

def set_attack(self, attack: int):
    self.attack = attack

def set_defense(self, defense: int):
    self.defense = defense

```

Client

```

def main():
    print("=== Creating Original NPC ===")
    alien = NPC("Alien", 30, 5, 2)
    alien.describe()

    print("\n=== Cloning NPC (No expensive operation) ===")
    alien_copy = alien.clone()
    alien_copy.describe()

    print("\n=== Cloning and Customizing ===")
    powerful_alien = alien.clone()
    powerful_alien.set_name("Powerful Alien")
    powerful_alien.set_health(50)
    powerful_alien.set_defense(5)
    powerful_alien.describe()

    print("\n=== Original is unchanged ===")
    alien.describe()

```

Alternative: Python's built-in copy module

```

class NPCPythonic:
    def __init__(self, name, health, attack, defense):
        print(f"Creating NPC: {name}")
        time.sleep(1)
        self.name = name
        self.health = health
        self.attack = attack
        self.defense = defense

    def clone(self):
        # Using Python's built-in deep copy
        return copy.deepcopy(self)

    def describe(self):
        print(f"NPC [name={self.name}, health={self.health}, "
              f"attack={self.attack}, defense={self.defense}]")

```

```
if __name__ == "__main__":
    main()
```

Python's Built-in copy Module

python

```
import copy

# Shallow copy - only copies top-level references
shallow = copy.copy(original)

# Deep copy - recursively copies all nested objects
deep = copy.deepcopy(original)

# For Python, __copy__() and __deepcopy__() can be overridden
class NPC:
    def __copy__(self):
        new_npc = NPC.__new__(NPC)
        new_npc.__dict__.update(self.__dict__)
        return new_npc

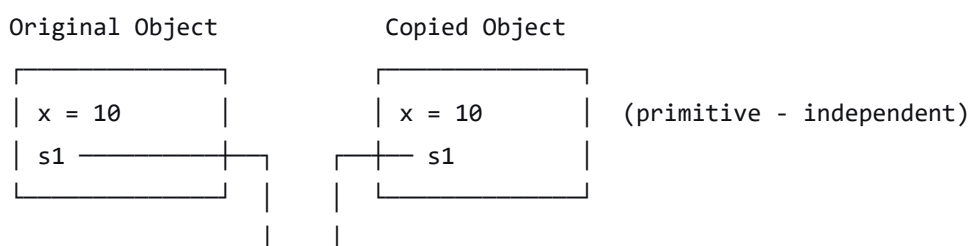
    def __deepcopy__(self, memo):
        new_npc = NPC.__new__(NPC)
        for key, value in self.__dict__.items():
            setattr(new_npc, key, copy.deepcopy(value, memo))
        return new_npc
```

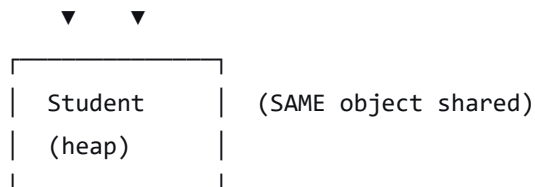
8. Shallow Copy vs Deep Copy

Shallow Copy

Copies **only top-level values**. For non-primitive types (objects/pointers), only the **reference** is copied — both objects point to the same underlying data.

text





Java code for shallow copy:

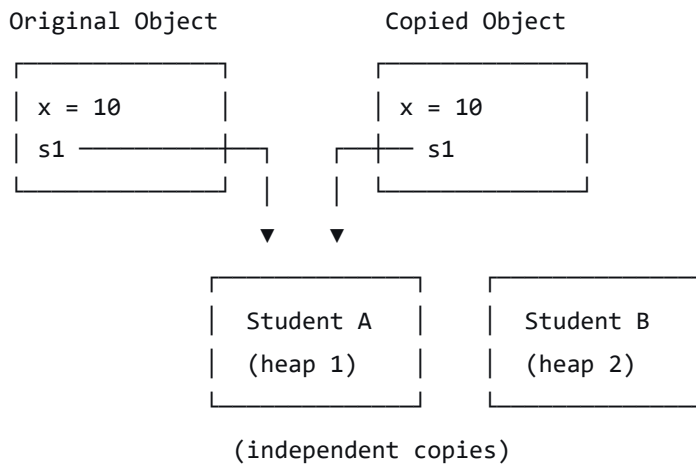
java

```
public NPC(NPC other) {
    this.name = other.name;    // String - OK (immutable, but still ref)
    this.health = other.health; // Primitive - OK, independent
    this.student = other.student; // Reference - SHARED! Problem!
}
```

Deep Copy

Copies **everything recursively**. Each object gets its own independent copy of nested objects.

text



Java code for deep copy:

java

```
public NPC(NPC other) {
    this.name = other.name;
    this.health = other.health;
    // Deep copy: create a NEW student object
    this.student = new Student(other.student);
}
```

Comparison Table

Aspect	Shallow Copy	Deep Copy
Primitives	Copied by value	Copied by value
Objects/References	Reference copied (shared)	Object is duplicated
Changes affect original?	Yes (for nested objects)	No
Speed	Faster	Slower
Memory	Less	More
When to use	Immutable nested objects	Mutable nested objects

Complete Example: Shallow vs Deep

```
java

class Student {
    String name;
    Student(String name) { this.name = name; }
    Student(Student other) { this.name = other.name; }
}

class NPC {
    int health;
    Student student; // Non-primitive

    // SHALLOW COPY - problematic
    public NPC shallowCopy(NPC other) {
        this.health = other.health;
        this.student = other.student; // Same reference!
        return this;
    }

    // DEEP COPY - safe
    public NPC deepCopy(NPC other) {
        this.health = other.health;
        this.student = new Student(other.student); // New object!
        return this;
    }
}
```

Python equivalent:

python

```
import copy
```

```
class Student:
```

```
    def __init__(self, name):  
        self.name = name
```

```
class NPC:
```

```
    def __init__(self, health, student):  
        self.health = health  
        self.student = student
```

```
    def shallow_copy(self, other):  
        self.health = other.health  
        self.student = other.student  # Same reference  
        return self
```

```
    def deep_copy(self, other):  
        self.health = other.health  
        self.student = copy.deepcopy(other.student)  # New object  
        return self
```

9. Advantages & Disadvantages

Advantages

- ✓ **Performance:** Avoids expensive object creation
- ✓ **Simplicity:** Copy is simpler than reconstructing
- ✓ **Reduced Subclassing:** No need for factory hierarchy
- ✓ **Dynamic Configuration:** Clone and customize at runtime
- ✓ **Memory Efficient:** For similar objects with minor differences

Disadvantages

- ✗ **Complex for Complex Objects:** Deep copy of complex graphs is tricky
- ✗ **Circular References:** Difficult to handle
- ✗ **Shallow vs Deep Confusion:** Easy to accidentally share state
- ✗ **Not for Very Different Objects:** If objects differ a lot, cloning is useless

10. When to Use Prototype Pattern

Use Prototype when:

1. **Object creation is expensive** (DB, network, heavy computation)
2. **You need many similar objects** with minor differences
3. **You want to avoid building a factory hierarchy** parallel to your class hierarchy
4. **Objects have few states** and are mostly similar
5. **Dynamic loading:** Classes are specified at runtime

Don't use when:

- Objects are very different from each other
- Object creation is cheap
- Objects have complex circular references
- Immutability is required

11. Real-World Use Cases

Use Case	Why Prototype?
Game NPCs	Many identical characters, expensive creation
Document Templates	Clone a template and customize
Configuration Objects	Clone default config, tweak a few fields
Cell Division Simulation	Biological cells clone themselves
Graph Editors	Clone shapes/groups of shapes
Spring Framework	Uses prototype scope for beans
Database Records	Clone loaded record for similar new one

12. Comparison with Related Patterns

Pattern	Key Difference from Prototype
Factory Method	Creates objects from scratch; Prototype clones existing
Abstract Factory	Creates families of objects; Prototype clones one object
Builder	Step-by-step construction; Prototype copies at once
Singleton	Ensures one instance; Prototype creates many from one

13. Key Takeaways

1. **Prototype = Clone instead of** `new`
2. **Use copy constructors** to implement cloning
3. **Marker interface** (like `Cloneable`) signals cloneability
4. **Shallow vs Deep copy** matters for non-primitive fields
5. **Best for:** Many similar objects, expensive creation
6. **Skip when:** Objects differ significantly or are cheap to create
7. **Real-world:** NPCs, templates, configuration objects, cells

14. Interview Tip

When discussing Prototype in interviews:

1. **State the problem:** "When object creation is expensive"
2. **Explain the solution:** "Clone an existing instance via copy constructor"
3. **Mention clone() method:** "Returns a copy of the object"
4. **Discuss shallow vs deep copy:** Show awareness of reference vs value copying
5. **Real examples:** NPCs in games, document templates, Spring prototype scope
6. **Compare with Factory:** Factory creates new; Prototype copies existing

Key phrase: "Prototype lets you create new objects by cloning an existing instance, avoiding expensive construction."



हां जी तो करें स्टार्ट। वेलकम बै...

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Chess.com Clone - Complete LLD Notes

1. Project Overview

Building an **online chess platform** (like [Chess.com](#)) where:

- Multiple users can play chess simultaneously
- Score-based matchmaking algorithm pairs players
- Standard chess rules with extensibility
- In-game chat between players (using Mediator pattern)
- Players can quit mid-game
- Score updates based on win/loss/draw

Prerequisites: Basic understanding of chess rules (how pieces move, check, checkmate, stalemate).

2. Functional Requirements

#	Requirement	Description
1	Multiple users playing simultaneously	Support concurrent matches; each user has a score (default 1000)
2	Score-based matchmaking	Pair users with closest score difference
3	Standard chess rules (extensible)	All standard rules; open for future custom rules
4	In-game chat	Users can send/receive messages during a match
5	User can quit match	Quitting = loss; winner gains points, quitter loses points

#	Requirement	Description
6	Score updates	Win: +points, Loss: -points, Draw: no change

3. Happy Flow

text

1. User U1 arrives at platform
2. U1 requests a match → added to waiting queue
3. Matchmaking algorithm compares U1's score with waiting users
4. Finds U2 with minimum score difference
5. Match created → separate "room" with:
 - Chess board (8x8)
 - Two players (White & Black)
 - Chat room (Mediator)
6. Players alternate turns; game continues
7. One player wins/loses/quits → game ends
8. Scores updated; users return to lobby

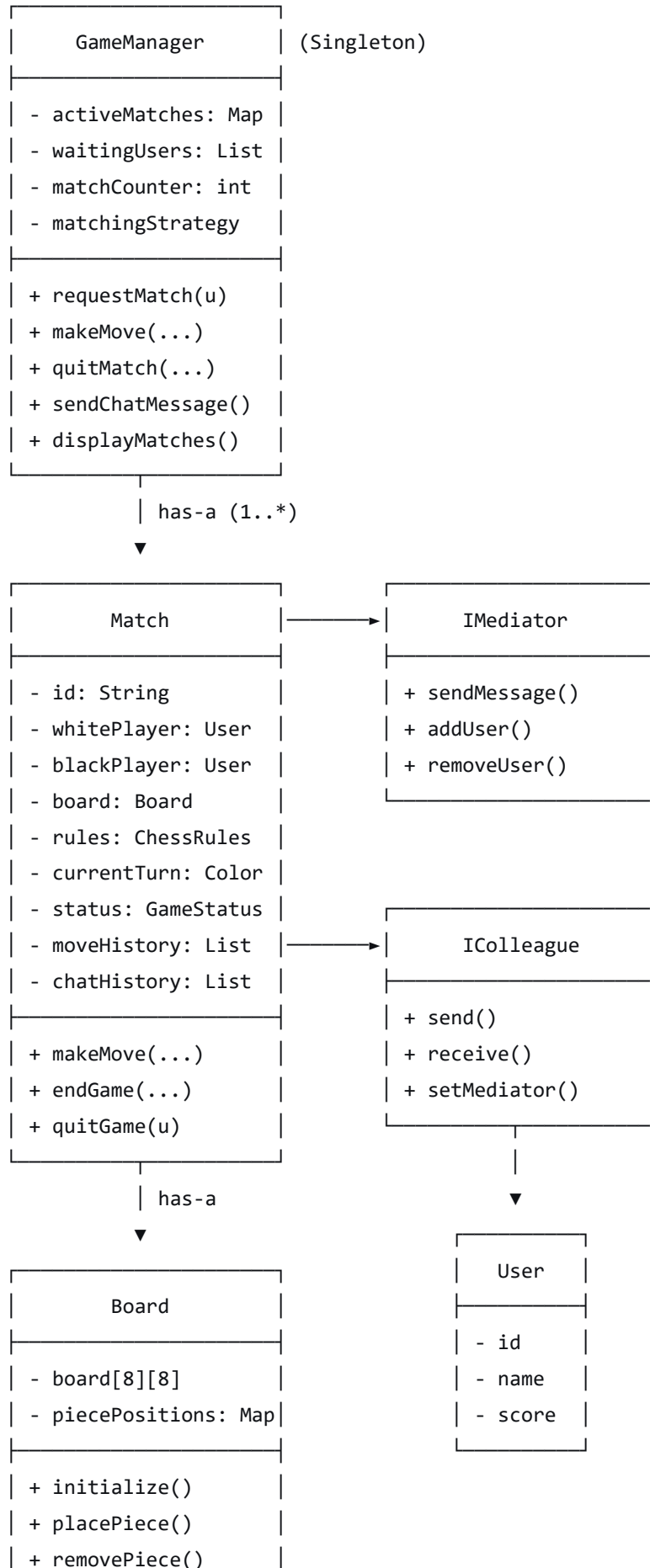
4. Top-Down Design Approach

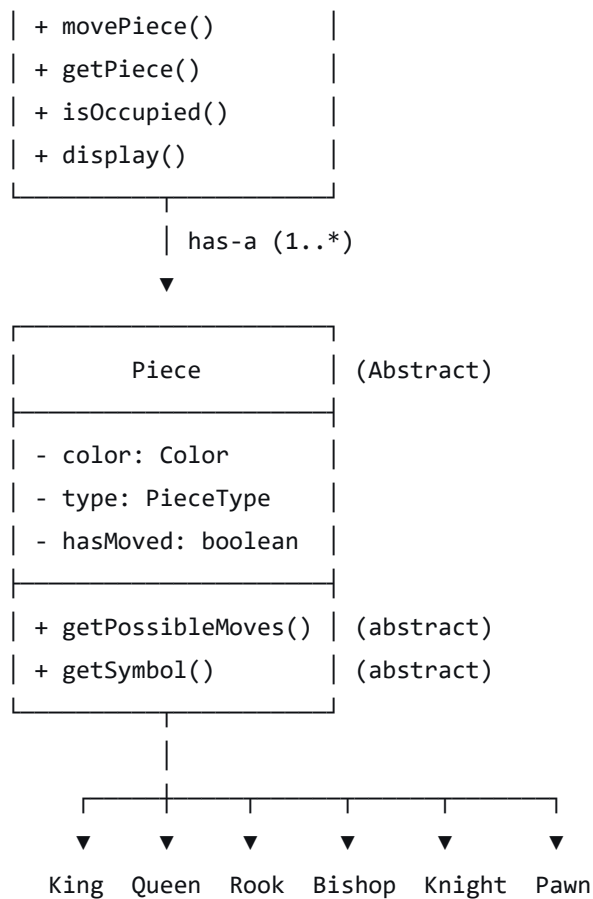
We design from the largest component down to smaller ones:

1. **GameManager** (orchestrator, Singleton)
2. **Match** (mediator for the two players)
3. **Board** (holds pieces)
4. **Piece** hierarchy (King, Queen, Rook, Bishop, Knight, Pawn)
5. **Position** (row, column)
6. **Move** (from, to, piece, captured piece)
7. **Rules** (ChessRules, StandardChessRules)
8. **User** (Colleague in Mediator pattern)
9. **Message** (chat)
10. **MatchingStrategy** (ScoreBasedMatching)

5. UML Class Design

text





6. Detailed Class Design

6.1 Enums

java

```

enum Color { WHITE, BLACK }
enum PieceType { KING, QUEEN, ROOK, BISHOP, KNIGHT, PAWN }
enum GameStatus { WAITING, IN_PROGRESS, COMPLETED, ABORTED }
  
```

6.2 Position

Holds row and column (0-7 for 8x8 board).

java

```

class Position {
    private int row;
    private int col;

    public Position(int row, int col) {
  
```

```

        this.row = row;
        this.col = col;
    }

    public boolean isValid() {
        return row >= 0 && row < 8 && col >= 0 && col < 8;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Position)) return false;
        Position p = (Position) o;
        return row == p.row && col == p.col;
    }

    @Override
    public int hashCode() {
        return 31 * row + col;
    }

    @Override
    public String toString() {
        char file = (char)('a' + col);
        int rank = 8 - row;
        return "" + file + rank;
    }
}

```

6.3 Move

java

```

class Move {
    private Position from;
    private Position to;
    private Piece piece;
    private Piece capturedPiece;

    public Move(Position from, Position to, Piece piece, Piece capturedPiece) {
        this.from = from;
        this.to = to;
        this.piece = piece;
        this.capturedPiece = capturedPiece;
    }
}

```

```

    // getters...
}

```

6.4 Piece (Abstract)

java

```

abstract class Piece {
    protected Color color;
    protected PieceType type;
    protected boolean hasMoved;

    public Piece(Color color, PieceType type) {
        this.color = color;
        this.type = type;
        this.hasMoved = false;
    }

    // Abstract methods to be overridden
    public abstract List<Position> getPossibleMoves(Position current, Board board);
    public abstract String getSymbol();

    public Color getColor() { return color; }
    public PieceType getType() { return type; }
    public boolean hasMoved() { return hasMoved; }
    public void setMoved(boolean moved) { this.hasMoved = moved; }
}

```

6.5 King

java

```

class King extends Piece {
    public King(Color color) {
        super(color, PieceType.KING);
    }

    @Override
    public List<Position> getPossibleMoves(Position current, Board board) {
        List<Position> moves = new ArrayList<>();
        int[][] directions = {
            {-1,-1}, {-1,0}, {-1,1},
            {0,-1},      {0,1},
            {1,-1},  {1,0},  {1,1}
        };
    }
}

```

```

    for (int[] d : directions) {
        Position next = new Position(current.getRow() + d[0],
                                     current.getCol() + d[1]);

        if (next.isValid() &&
            !board.isOccupiedBySameColor(next, this.color)) {
            moves.add(next);
        }
    }
    return moves;
}

@Override
public String getSymbol() { return "K"; }
}

```

6.6 Queen

java

```

class Queen extends Piece {
    public Queen(Color color) {
        super(color, PieceType.QUEEN);
    }

    @Override
    public List<Position> getPossibleMoves(Position current, Board board) {
        List<Position> moves = new ArrayList<>();
        int[][] directions = {
            {-1,-1}, {-1,0}, {-1,1},
            {0,-1},      {0,1},
            {1,-1},  {1,0},  {1,1}
        };

        // Queen slides in all 8 directions until blocked
        for (int[] d : directions) {
            for (int i = 1; i < 8; i++) {
                Position next = new Position(current.getRow() + d[0]*i,
                                             current.getCol() + d[1]*i);

                if (!next.isValid()) break;
                if (board.isOccupiedBySameColor(next, this.color)) break;
                moves.add(next);
                if (board.isOccupied(next)) break; // captured enemy piece
            }
        }
        return moves;
    }
}

```

```

@Override
public String getSymbol() { return "Q"; }
}

```

6.7 Rook

java

```

class Rook extends Piece {
    public Rook(Color color) {
        super(color, PieceType.ROOK);
    }

    @Override
    public List<Position> getPossibleMoves(Position current, Board board) {
        List<Position> moves = new ArrayList<>();
        int[][] directions = {{-1,0}, {1,0}, {0,-1}, {0,1}};

        for (int[] d : directions) {
            for (int i = 1; i < 8; i++) {
                Position next = new Position(current.getRow() + d[0]*i,
                                                current.getCol() + d[1]*i);

                if (!next.isValid()) break;
                if (board.isOccupiedBySameColor(next, this.color)) break;
                moves.add(next);
                if (board.isOccupied(next)) break;
            }
        }
        return moves;
    }

    @Override
    public String getSymbol() { return "R"; }
}

```

6.8 Bishop

java

```

class Bishop extends Piece {
    public Bishop(Color color) {
        super(color, PieceType.BISHOP);
    }
}

```

```

@Override
public List<Position> getPossibleMoves(Position current, Board board) {
    List<Position> moves = new ArrayList<>();
    int[][] directions = {{-1,-1}, {-1,1}, {1,-1}, {1,1}};

    for (int[] d : directions) {
        for (int i = 1; i < 8; i++) {
            Position next = new Position(current.getRow() + d[0]*i,
                                         current.getCol() + d[1]*i);

            if (!next.isValid()) break;
            if (board.isOccupiedBySameColor(next, this.color)) break;
            moves.add(next);
            if (board.isOccupied(next)) break;
        }
    }
    return moves;
}

@Override
public String getSymbol() { return "B"; }
}

```

6.9 Knight

java

```

class Knight extends Piece {
    public Knight(Color color) {
        super(color, PieceType.KNIGHT);
    }

    @Override
    public List<Position> getPossibleMoves(Position current, Board board) {
        List<Position> moves = new ArrayList<>();
        int[][] offsets = {
            {-2,-1}, {-2,1}, {-1,-2}, {-1,2},
            {1,-2}, {1,2}, {2,-1}, {2,1}
        };

        for (int[] o : offsets) {
            Position next = new Position(current.getRow() + o[0],
                                         current.getCol() + o[1]);

            if (next.isValid() &&
                !board.isOccupiedBySameColor(next, this.color)) {
                moves.add(next);
            }
        }
    }
}

```

```

        return moves;
    }

    @Override
    public String getSymbol() { return "N"; }
}

```

6.10 Pawn

java

```

class Pawn extends Piece {
    public Pawn(Color color) {
        super(color, PieceType.PAWN);
    }

    @Override
    public List<Position> getPossibleMoves(Position current, Board board) {
        List<Position> moves = new ArrayList<>();
        int direction = (color == Color.WHITE) ? -1 : 1;

        // Forward one step
        Position oneStep = new Position(current.getRow() + direction, current.getCol());
        if (oneStep.isValid() && !board.isOccupied(oneStep)) {
            moves.add(oneStep);

            // Forward two steps (only if not moved yet)
            if (!hasMoved) {
                Position twoStep = new Position(current.getRow() + 2*direction,
                                                  current.getCol());
                if (twoStep.isValid() && !board.isOccupied(twoStep)) {
                    moves.add(twoStep);
                }
            }
        }

        // Diagonal captures
        int[] colOffsets = {-1, 1};
        for (int cOff : colOffsets) {
            Position diag = new Position(current.getRow() + direction,
                                          current.getCol() + cOff);
            if (diag.isValid() && board.isOccupied(diag) &&
                !board.isOccupiedBySameColor(diag, this.color)) {
                moves.add(diag);
            }
        }
    }
}

```

```

        return moves;
    }

    @Override
    public String getSymbol() { return "P"; }
}

```

6.11 PieceFactory

java

```

class PieceFactory {
    public static Piece createPiece(PieceType type, Color color) {
        switch (type) {
            case KING: return new King(color);
            case QUEEN: return new Queen(color);
            case ROOK: return new Rook(color);
            case BISHOP: return new Bishop(color);
            case KNIGHT: return new Knight(color);
            case PAWN: return new Pawn(color);
            default: throw new IllegalArgumentException("Unknown type");
        }
    }
}

```

6.12 Board

java

```

class Board {
    private Piece[][] grid; // 8x8 grid
    private Map<Position, Piece> piecePositions; // position → piece

    public Board() {
        grid = new Piece[8][8];
        piecePositions = new HashMap<>();
        initializeBoard();
    }

    private void initializeBoard() {
        // White pieces (bottom rows)
        placePiece(new Position(7, 0), new Rook(Color.WHITE));
        placePiece(new Position(7, 1), new Knight(Color.WHITE));
        placePiece(new Position(7, 2), new Bishop(Color.WHITE));
        placePiece(new Position(7, 3), new Queen(Color.WHITE));
    }
}

```

```

placePiece(new Position(7, 4), new King(Color.WHITE));
placePiece(new Position(7, 5), new Bishop(Color.WHITE));
placePiece(new Position(7, 6), new Knight(Color.WHITE));
placePiece(new Position(7, 7), new Rook(Color.WHITE));
for (int c = 0; c < 8; c++) {
    placePiece(new Position(6, c), new Pawn(Color.WHITE));
}

// Black pieces (top rows)
placePiece(new Position(0, 0), new Rook(Color.BLACK));
placePiece(new Position(0, 1), new Knight(Color.BLACK));
placePiece(new Position(0, 2), new Bishop(Color.BLACK));
placePiece(new Position(0, 3), new Queen(Color.BLACK));
placePiece(new Position(0, 4), new King(Color.BLACK));
placePiece(new Position(0, 5), new Bishop(Color.BLACK));
placePiece(new Position(0, 6), new Knight(Color.BLACK));
placePiece(new Position(0, 7), new Rook(Color.BLACK));
for (int c = 0; c < 8; c++) {
    placePiece(new Position(1, c), new Pawn(Color.BLACK));
}
}

public void placePiece(Position pos, Piece piece) {
    grid[pos.getRow()][pos.getCol()] = piece;
    piecePositions.put(pos, piece);
}

public void removePiece(Position pos) {
    Piece p = grid[pos.getRow()][pos.getCol()];
    if (p != null) {
        grid[pos.getRow()][pos.getCol()] = null;
        piecePositions.remove(pos);
    }
}

public Piece getPiece(Position pos) {
    if (!pos.isValid()) return null;
    return grid[pos.getRow()][pos.getCol()];
}

public boolean isOccupied(Position pos) {
    return getPiece(pos) != null;
}

public boolean isOccupiedBySameColor(Position pos, Color color) {
    Piece p = getPiece(pos);
    return p != null && p.getColor() == color;
}

```

```

public void movePiece(Position from, Position to) {
    Piece piece = getPiece(from);
    if (piece == null) return;

    // Handle capture
    Piece captured = getPiece(to);
    if (captured != null) {
        removePiece(to);
    }

    // Move
    removePiece(from);
    placePiece(to, piece);
    piece.setMoved(true);
}

public Position findKing(Color color) {
    for (Map.Entry<Position, Piece> e : piecePositions.entrySet()) {
        Piece p = e.getValue();
        if (p.getType() == PieceType.KING && p.getColor() == color) {
            return e.getKey();
        }
    }
    return new Position(-1, -1);
}

public List<Position> getAllPiecesOfColor(Color color) {
    List<Position> result = new ArrayList<>();
    for (Map.Entry<Position, Piece> e : piecePositions.entrySet()) {
        if (e.getValue().getColor() == color) {
            result.add(e.getKey());
        }
    }
    return result;
}

public void display() {
    // Pretty print board
    System.out.println(" a b c d e f g h");
    for (int r = 0; r < 8; r++) {
        System.out.print((8 - r) + " ");
        for (int c = 0; c < 8; c++) {
            Piece p = grid[r][c];
            if (p == null) System.out.print(".. ");
            else System.out.print(p.getColor() == Color.WHITE ? "W" : "B");
            if (p != null) System.out.print(p.getSymbol() + " ");
            else System.out.print(" ");
        }
    }
}

```

```

    }
    System.out.println();
}
}
}

```

6.13 ChessRules (Abstract)

java

```

abstract class ChessRules {
    public abstract boolean isValidMove(Move move, Board board);
    public abstract boolean isInCheck(Color color, Board board);
    public abstract boolean isCheckmate(Color color, Board board);
    public abstract boolean isStalemate(Color color, Board board);
    public abstract boolean wouldMoveCauseCheck(Move move, Board board, Color color);
}

```

6.14 StandardChessRules

java

```

class StandardChessRules extends ChessRules {

    @Override
    public boolean isValidMove(Move move, Board board) {
        Piece piece = board.getPiece(move.getFrom());
        if (piece == null) return false;

        // Check if the target position is in the piece's possible moves
        List<Position> possible = piece.getPossibleMoves(move.getFrom(), board);
        boolean validDestination = false;
        for (Position p : possible) {
            if (p.equals(move.getTo())) {
                validDestination = true;
                break;
            }
        }
        if (!validDestination) return false;

        // Check if this move would put own king in check
        if (wouldMoveCauseCheck(move, board, piece.getColor())) {
            return false;
        }
    }
}

```

```

        return true;
    }

    @Override
    public boolean isInCheck(Color color, Board board) {
        Position kingPos = board.findKing(color);
        if (kingPos.getRow() == -1) return false;

        Color opponentColor = (color == Color.WHITE) ? Color.BLACK : Color.WHITE;
        List<Position> opponentPieces = board.getAllPiecesOfColor(opponentColor);

        for (Position p : opponentPieces) {
            Piece piece = board.getPiece(p);
            List<Position> moves = piece.getPossibleMoves(p, board);
            for (Position target : moves) {
                if (target.equals(kingPos)) {
                    return true;
                }
            }
        }
        return false;
    }

    @Override
    public boolean isCheckmate(Color color, Board board) {
        if (!isInCheck(color, board)) return false;

        // Try every possible move for this color
        List<Position> pieces = board.getAllPiecesOfColor(color);
        for (Position p : pieces) {
            Piece piece = board.getPiece(p);
            List<Position> moves = piece.getPossibleMoves(p, board);
            for (Position target : moves) {
                Move testMove = new Move(p, target, piece, board.getPiece(target));
                if (isValidMove(testMove, board)) {
                    return false; // found at least one valid escape
                }
            }
        }
        return true;
    }

    @Override
    public boolean isStalemate(Color color, Board board) {
        if (isInCheck(color, board)) return false;

        List<Position> pieces = board.getAllPiecesOfColor(color);
        for (Position p : pieces) {

```

```

        Piece piece = board.getPiece(p);
        List<Position> moves = piece.getPossibleMoves(p, board);
        for (Position target : moves) {
            Move testMove = new Move(p, target, piece, board.getPiece(target));
            if (isValidMove(testMove, board)) {
                return false;
            }
        }
    }
    return true;
}

@Override
public boolean wouldMoveCauseCheck(Move move, Board board, Color color) {
    // Temporarily execute the move
    Piece movingPiece = board.getPiece(move.getFrom());
    Piece capturedPiece = board.getPiece(move.getTo());

    board.removePiece(move.getFrom());
    if (capturedPiece != null) board.removePiece(move.getTo());
    board.placePiece(move.getTo(), movingPiece);

    boolean inCheck = isInCheck(color, board);

    // Undo the move
    board.removePiece(move.getTo());
    board.placePiece(move.getFrom(), movingPiece);
    if (capturedPiece != null) board.placePiece(move.getTo(), capturedPiece);

    return inCheck;
}
}

```

6.15 Mediator + Colleague (for Chat)

java

```

interface IMediator {
    void sendMessage(Message message, User fromUser);
    void addUser(User user);
    void removeUser(User user);
}

abstract class IColleague {
    protected IMediator mediator;

    public void setMediator(IMediator mediator) {

```

```

        this.mediator = mediator;
    }

    public abstract void send(Message message);
    public abstract void receive(Message message);
}

```

6.16 User (Colleague)

java

```

class User extends IColleague {
    private String id;
    private String name;
    private int score;

    public User(String id, String name, int score) {
        this.id = id;
        this.name = name;
        this.score = score;
    }

    @Override
    public void send(Message message) {
        if (mediator != null) {
            mediator.sendMessage(message, this);
        }
    }

    @Override
    public void receive(Message message) {
        System.out.println(name + " received: " + message.getContent());
    }

    public void incrementScore(int delta) { score += delta; }
    public void decrementScore(int delta) { score -= delta; }

    public String getId() { return id; }
    public String getName() { return name; }
    public int getScore() { return score; }
}

```

6.17 Message

java

```
class Message {
    private String senderId;
    private String content;
    private long timestamp;

    public Message(String senderId, String content) {
        this.senderId = senderId;
        this.content = content;
        this.timestamp = System.currentTimeMillis();
    }

    public String getSenderId() { return senderId; }
    public String getContent() { return content; }
    public long getTimestamp() { return timestamp; }
}
```

6.18 Match (Concrete Mediator)

java

```
class Match implements IMediator {
    private String id;
    private User whitePlayer;
    private User blackPlayer;
    private Board board;
    private ChessRules rules;
    private Color currentTurn;
    private GameStatus status;
    private List<Move> moveHistory;
    private List<Message> chatHistory;

    public Match(String id, User whitePlayer, User blackPlayer) {
        this.id = id;
        this.whitePlayer = whitePlayer;
        this.blackPlayer = blackPlayer;
        this.board = new Board();
        this.rules = new StandardChessRules();
        this.currentTurn = Color.WHITE;
        this.status = GameStatus.IN_PROGRESS;
        this.moveHistory = new ArrayList<>();
        this.chatHistory = new ArrayList<>();

        // Register both users as colleagues
        whitePlayer.setMediator(this);
        blackPlayer.setMediator(this);
    }
}
```

```
        System.out.println("Match started between " + whitePlayer.getName()
            + " (White) and " + blackPlayer.getName() + " (Black)");
    }

    public boolean makeMove(Position from, Position to, User player) {
        // Validations
        if (status != GameStatus.IN_PROGRESS) {
            System.out.println("Game is not in progress");
            return false;
        }

        Color playerColor = getPlayerColor(player);
        if (playerColor != currentTurn) {
            System.out.println("Not your turn!");
            return false;
        }

        Piece piece = board.getPiece(from);
        if (piece == null || piece.getColor() != playerColor) {
            System.out.println("Invalid piece selection");
            return false;
        }

        // Create move
        Move move = new Move(from, to, piece, board.getPiece(to));

        // Validate
        if (!rules.isValidMove(move, board)) {
            System.out.println("Invalid move");
            return false;
        }

        // Execute
        board.movePiece(from, to);
        moveHistory.add(move);
        board.display();

        // Check game conditions
        Color opponent = (playerColor == Color.WHITE) ? Color.BLACK : Color.WHITE;

        if (rules.isCheckmate(opponent, board)) {
            endGame(player, "CHECKMATE");
            return true;
        }

        if (rules.isStalemate(opponent, board)) {
            endGame(player, "STALEMATE");
            return true;
        }
    }
}
```

```

    }

    // Switch turn
    currentTurn = opponent;

    if (rules.isInCheck(opponent, board)) {
        System.out.println("Opponent is in check!");
    }

    return true;
}

public void quitGame(User player) {
    Color playerColor = getPlayerColor(player);
    User opponent = (player == whitePlayer) ? blackPlayer : whitePlayer;
    endGame(opponent, "QUIT");
    player.decrementScore(50);
}

public void endGame(User winner, String reason) {
    status = GameStatus.COMPLETED;
    System.out.println("Game ended. " + winner.getName()
        + " wins by " + reason);

    if (winner != null) {
        User loser = (winner == whitePlayer) ? blackPlayer : whitePlayer;
        winner.incrementScore(30);
        loser.decrementScore(20);
        System.out.println("Scores updated.");
    }
}

private Color getPlayerColor(User player) {
    return (player == whitePlayer) ? Color.WHITE : Color.BLACK;
}

// Mediator methods
@Override
public void sendMessage(Message message, User fromUser) {
    chatHistory.add(message);
    User recipient = (fromUser == whitePlayer) ? blackPlayer : whitePlayer;
    recipient.receive(message);
}

@Override
public void addUser(User user) {
    // Chess has exactly 2 users; do nothing or throw
    throw new UnsupportedOperationException("Cannot add more users");
}

```

```

    }

    @Override
    public void removeUser(User user) {
        quitGame(user);
    }

    public String getId() { return id; }
    public GameStatus getStatus() { return status; }
    public Board getBoard() { return board; }
    public User getWhitePlayer() { return whitePlayer; }
    public User getBlackPlayer() { return blackPlayer; }
}

```

6.19 MatchingStrategy (Abstract)

java

```

abstract class MatchingStrategy {
    public abstract User findMatch(User user, List<User> waitingUsers);
}

```

6.20 ScoreBasedMatching (Strategy)

java

```

class ScoreBasedMatching extends MatchingStrategy {
    private int scoreTolerance;

    public ScoreBasedMatching(int scoreTolerance) {
        this.scoreTolerance = scoreTolerance;
    }

    @Override
    public User findMatch(User user, List<User> waitingUsers) {
        User bestMatch = null;
        int bestScoreDiff = Integer.MAX_VALUE;

        for (User candidate : waitingUsers) {
            if (candidate.getId().equals(user.getId())) continue;

            int diff = Math.abs(candidate.getScore() - user.getScore());
            if (diff <= scoreTolerance && diff < bestScoreDiff) {
                bestScoreDiff = diff;
                bestMatch = candidate;
            }
        }

        return bestMatch;
    }
}

```

```

    }
}
return bestMatch;
}
}

```

6.21 GameManager (Singleton - Orchestrator)

java

```

class GameManager {
    private static GameManager instance;

    private Map<String, Match> activeMatches;
    private List<User> waitingUsers;
    private int matchCounter;
    private MatchingStrategy matchingStrategy;

    private GameManager() {
        activeMatches = new HashMap<>();
        waitingUsers = new ArrayList<>();
        matchCounter = 0;
        matchingStrategy = new ScoreBasedMatching(100);
    }

    public static synchronized GameManager getInstance() {
        if (instance == null) {
            instance = new GameManager();
        }
        return instance;
    }

    public Match requestMatch(User user) {
        System.out.println(user.getName() + " is looking for a match");

        User opponent = matchingStrategy.findMatch(user, waitingUsers);

        if (opponent == null) {
            waitingUsers.add(user);
            System.out.println(user.getName() + " added to waiting list");
            return null;
        }

        waitingUsers.remove(opponent);
        matchCounter++;
        String matchId = "MATCH_" + matchCounter;
    }
}

```

```

        Match match = new Match(matchId, user, opponent);
        activeMatches.put(matchId, match);
        System.out.println("Match found: " + matchId);
        match.getBoard().display();
        return match;
    }

    public boolean makeMove(String matchId, Position from, Position to, User player) {
        Match match = activeMatches.get(matchId);
        if (match == null) {
            System.out.println("Match not found");
            return false;
        }

        boolean success = match.makeMove(from, to, player);

        if (match.getStatus() == GameStatus.COMPLETED) {
            activeMatches.remove(matchId);
            System.out.println("Match " + matchId + " completed and removed");
        }
        return success;
    }

    public void quitMatch(String matchId, User player) {
        Match match = activeMatches.get(matchId);
        if (match == null) return;
        match.quitGame(player);
        activeMatches.remove(matchId);
    }

    public void sendChatMessage(String matchId, String content, User user) {
        Match match = activeMatches.get(matchId);
        if (match == null) return;
        Message msg = new Message(user.getId(), content);
        match.sendMessage(msg, user);
    }

    public void displayActiveMatches() {
        System.out.println("Total active matches: " + activeMatches.size());
        System.out.println("Users waiting: " + waitingUsers.size());
        for (Map.Entry<String, Match> e : activeMatches.entrySet()) {
            System.out.println("  " + e.getKey() + ": "
                + e.getValue().getWhitePlayer().getName() + " vs "
                + e.getValue().getBlackPlayer().getName());
        }
    }
}

```

6.22 Main + Scholar's Mate Demo

java

```
public class Main {
    public static void main(String[] args) {
        scholarMateDemo();
        System.out.println("\n===== MATCHMAKING DEMO =====");
        matchMakingDemo();
    }

    // Scholar's Mate: 4-move checkmate
    static void scholarMateDemo() {
        User aditya = new User("DEMO_1", "Aditya", 1000);
        User rohit = new User("DEMO_2", "Rohit", 1000);

        Match match = new Match("DEMO_MATCH", aditya, rohit);
        match.getBoard().display();

        // Move 1: White pawn e2-e4
        match.makeMove(new Position(6, 4), new Position(4, 4), aditya);
        // Black pawn e7-e5
        match.makeMove(new Position(1, 4), new Position(3, 4), rohit);

        // Move 2: White bishop f1-c4
        match.makeMove(new Position(7, 5), new Position(4, 2), aditya);
        // Black knight b8-c6
        match.makeMove(new Position(0, 1), new Position(2, 2), rohit);

        // Move 3: White queen d1-h5
        match.makeMove(new Position(7, 3), new Position(3, 7), aditya);
        // Black knight g8-f6
        match.makeMove(new Position(0, 6), new Position(2, 5), rohit);

        // Move 4: White queen h5-f7 → checkmate
        match.makeMove(new Position(3, 7), new Position(1, 5), aditya);

        // Chat
        match.sendMessage(new Message(aditya.getId(), "Good game!"), aditya);
        match.sendMessage(new Message(rohit.getId(), "That was quick!"), rohit);
    }

    static void matchMakingDemo() {
        GameManager gm = GameManager.getInstance();

        User sourav = new User("U1", "Sourav", 1000);
        User manish = new User("U2", "Manish", 1000);
        User abhishek = new User("U3", "Abhishek", 1000);
    }
}
```

```
gm.requestMatch(sourav);    // goes to waiting
gm.requestMatch(manish);    // matches with Sourav
gm.requestMatch(abhishek);  // goes to waiting

gm.displayActiveMatches();

}
```

7. Python Implementation (Core Parts)

python

```
from abc import ABC, abstractmethod
from enum import Enum
from typing import List, Optional
from dataclasses import dataclass
```

```
class Color(Enum):
    WHITE = 1
    BLACK = 2
```

```
class PieceType(Enum):
    KING = 1
    QUEEN = 2
    ROOK = 3
    BISHOP = 4
    KNIGHT = 5
    PAWN = 6
```

```
class GameStatus(Enum):
    WAITING = 1
    IN_PROGRESS = 2
    COMPLETED = 3
    ABORTED = 4
```

```
class Position:
    def __init__(self, row: int, col: int):
        self.row = row
        self.col = col
```

```

def is_valid(self) -> bool:
    return 0 <= self.row < 8 and 0 <= self.col < 8

def __eq__(self, other):
    return (isinstance(other, Position) and
            self.row == other.row and self.col == other.col)

def __hash__(self):
    return hash((self.row, self.col))

def __str__(self):
    return f"{chr(ord('a') + self.col)}{8 - self.row}"

@dataclass
class Move:
    from_pos: Position
    to_pos: Position
    piece: 'Piece'
    captured_piece: Optional['Piece']

class Piece(ABC):
    def __init__(self, color: Color, ptype: PieceType):
        self.color = color
        self.type = ptype
        self.has_moved = False

    @abstractmethod
    def get_possible_moves(self, current: Position, board: 'Board') -> List[Position]:
        pass

    @abstractmethod
    def get_symbol(self) -> str:
        pass

class King(Piece):
    def __init__(self, color):
        super().__init__(color, PieceType.KING)

    def get_possible_moves(self, current, board):
        moves = []
        directions = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]
        for dr, dc in directions:
            nxt = Position(current.row + dr, current.col + dc)
            if nxt.is_valid() and not board.is_occupied_by_same_color(nxt, self.color):
                moves.append(nxt)

```

```

        return moves

    def get_symbol(self):
        return "K"

class Queen(Piece):
    def __init__(self, color):
        super().__init__(color, PieceType.QUEEN)

    def get_possible_moves(self, current, board):
        moves = []
        directions = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]
        for dr, dc in directions:
            for i in range(1, 8):
                nxt = Position(current.row + dr*i, current.col + dc*i)
                if not nxt.is_valid():
                    break
                if board.is_occupied_by_same_color(nxt, self.color):
                    break
                moves.append(nxt)
                if board.is_occupied(nxt):
                    break
            return moves

    def get_symbol(self):
        return "Q"

class Rook(Piece):
    def __init__(self, color):
        super().__init__(color, PieceType.ROOK)

    def get_possible_moves(self, current, board):
        moves = []
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            for i in range(1, 8):
                nxt = Position(current.row + dr*i, current.col + dc*i)
                if not nxt.is_valid():
                    break
                if board.is_occupied_by_same_color(nxt, self.color):
                    break
                moves.append(nxt)
                if board.is_occupied(nxt):
                    break
            return moves

    def get_symbol(self):

```

```
return "R"
```

```
class Bishop(Piece):
    def __init__(self, color):
        super().__init__(color, PieceType.BISHOP)

    def get_possible_moves(self, current, board):
        moves = []
        for dr, dc in [(-1,-1),(-1,1),(1,-1),(1,1)]:
            for i in range(1, 8):
                nxt = Position(current.row + dr*i, current.col + dc*i)
                if not nxt.is_valid():
                    break
                if board.is_occupied_by_same_color(nxt, self.color):
                    break
                moves.append(nxt)
                if board.is_occupied(nxt):
                    break
            return moves

    def get_symbol(self):
        return "B"

class Knight(Piece):
    def __init__(self, color):
        super().__init__(color, PieceType.KNIGHT)

    def get_possible_moves(self, current, board):
        moves = []
        offsets = [(-2,-1),(-2,1),(-1,-2),(-1,2),(1,-2),(1,2),(2,-1),(2,1)]
        for dr, dc in offsets:
            nxt = Position(current.row + dr, current.col + dc)
            if nxt.is_valid() and not board.is_occupied_by_same_color(nxt, self.color):
                moves.append(nxt)
        return moves

    def get_symbol(self):
        return "N"

class Pawn(Piece):
    def __init__(self, color):
        super().__init__(color, PieceType.PAWN)

    def get_possible_moves(self, current, board):
        moves = []
```

```

direction = -1 if self.color == Color.WHITE else 1

# Forward 1
one = Position(current.row + direction, current.col)
if one.is_valid() and not board.is_occupied(one):
    moves.append(one)
    # Forward 2 (first move only)
    if not self.has_moved:
        two = Position(current.row + 2*direction, current.col)
        if two.is_valid() and not board.is_occupied(two):
            moves.append(two)

# Diagonal captures
for dc in [-1, 1]:
    diag = Position(current.row + direction, current.col + dc)
    if diag.is_valid() and board.is_occupied(diag) and \
        not board.is_occupied_by_same_color(diag, self.color):
        moves.append(diag)

return moves

def get_symbol(self):
    return "P"

class PieceFactory:
    @staticmethod
    def create(ptype: PieceType, color: Color) -> Piece:
        return {
            PieceType.KING: King,
            PieceType.QUEEN: Queen,
            PieceType.ROOK: Rook,
            PieceType.BISHOP: Bishop,
            PieceType.KNIGHT: Knight,
            PieceType.PAWN: Pawn,
        }[ptype](color)

class Board:
    def __init__(self):
        self.grid = [[None]*8 for _ in range(8)]
        self.piece_positions = {}
        self._initialize()

    def _initialize(self):
        # White
        self.place_piece(Position(7,0), Rook(Color.WHITE))
        self.place_piece(Position(7,1), Knight(Color.WHITE))

```

```

self.place_piece(Position(7,2), Bishop(Color.WHITE))
self.place_piece(Position(7,3), Queen(Color.WHITE))
self.place_piece(Position(7,4), King(Color.WHITE))
self.place_piece(Position(7,5), Bishop(Color.WHITE))
self.place_piece(Position(7,6), Knight(Color.WHITE))
self.place_piece(Position(7,7), Rook(Color.WHITE))
for c in range(8):
    self.place_piece(Position(6,c), Pawn(Color.WHITE))

# Black
self.place_piece(Position(0,0), Rook(Color.BLACK))
self.place_piece(Position(0,1), Knight(Color.BLACK))
self.place_piece(Position(0,2), Bishop(Color.BLACK))
self.place_piece(Position(0,3), Queen(Color.BLACK))
self.place_piece(Position(0,4), King(Color.BLACK))
self.place_piece(Position(0,5), Bishop(Color.BLACK))
self.place_piece(Position(0,6), Knight(Color.BLACK))
self.place_piece(Position(0,7), Rook(Color.BLACK))
for c in range(8):
    self.place_piece(Position(1,c), Pawn(Color.BLACK))

def place_piece(self, pos: Position, piece: Piece):
    self.grid[pos.row][pos.col] = piece
    self.piece_positions[pos] = piece

def remove_piece(self, pos: Position):
    if self.grid[pos.row][pos.col]:
        self.grid[pos.row][pos.col] = None
        self.piece_positions.pop(pos, None)

def get_piece(self, pos: Position) -> Optional[Piece]:
    if not pos.is_valid():
        return None
    return self.grid[pos.row][pos.col]

def is_occupied(self, pos: Position) -> bool:
    return self.get_piece(pos) is not None

def is_occupied_by_same_color(self, pos: Position, color: Color) -> bool:
    p = self.get_piece(pos)
    return p is not None and p.color == color

def move_piece(self, from_pos: Position, to_pos: Position):
    piece = self.get_piece(from_pos)
    if piece is None:
        return
    captured = self.get_piece(to_pos)
    if captured is not None:

```

```

        self.remove_piece(to_pos)
        self.remove_piece(from_pos)
        self.place_piece(to_pos, piece)
        piece.has_moved = True

def find_king(self, color: Color) -> Position:
    for pos, piece in self.piece_positions.items():
        if piece.type == PieceType.KING and piece.color == color:
            return pos
    return Position(-1, -1)

def get_all_pieces_of_color(self, color: Color) -> List[Position]:
    return [pos for pos, p in self.piece_positions.items() if p.color == color]

def display(self):
    print(" a b c d e f g h")
    for r in range(8):
        row = f"{8 - r} "
        for c in range(8):
            p = self.grid[r][c]
            if p is None:
                row += ".. "
            else:
                prefix = "W" if p.color == Color.WHITE else "B"
                row += prefix + p.get_symbol() + " "
        print(row)

class ChessRules(ABC):
    @abstractmethod
    def is_valid_move(self, move: Move, board: Board) -> bool: pass

    @abstractmethod
    def is_in_check(self, color: Color, board: Board) -> bool: pass

    @abstractmethod
    def is_checkmate(self, color: Color, board: Board) -> bool: pass

    @abstractmethod
    def is_stalemate(self, color: Color, board: Board) -> bool: pass

class StandardChessRules(ChessRules):
    def is_valid_move(self, move, board):
        piece = board.get_piece(move.from_pos)
        if piece is None:
            return False
        possible = piece.get_possible_moves(move.from_pos, board)

```

```
if not any(p == move.to_pos for p in possible):
    return False
if self.would_move_cause_check(move, board, piece.color):
    return False
return True

def is_in_check(self, color, board):
    king_pos = board.find_king(color)
    if king_pos.row == -1:
        return False
    opponent = Color.BLACK if color == Color.WHITE else Color.WHITE
    for p in board.get_all_pieces_of_color(opponent):
        piece = board.get_piece(p)
        for target in piece.get_possible_moves(p, board):
            if target == king_pos:
                return True
    return False

def is_checkmate(self, color, board):
    if not self.is_in_check(color, board):
        return False
    for p in board.get_all_pieces_of_color(color):
        piece = board.get_piece(p)
        for target in piece.get_possible_moves(p, board):
            move = Move(p, target, piece, board.get_piece(target))
            if self.is_valid_move(move, board):
                return False
    return True

def is_stalemate(self, color, board):
    if self.is_in_check(color, board):
        return False
    for p in board.get_all_pieces_of_color(color):
        piece = board.get_piece(p)
        for target in piece.get_possible_moves(p, board):
            move = Move(p, target, piece, board.get_piece(target))
            if self.is_valid_move(move, board):
                return False
    return True

def would_move_cause_check(self, move, board, color):
    moving = board.get_piece(move.from_pos)
    captured = board.get_piece(move.to_pos)
    board.remove_piece(move.from_pos)
    if captured: board.remove_piece(move.to_pos)
    board.place_piece(move.to_pos, moving)

    in_check = self.is_in_check(color, board)
```

```

        board.remove_piece(move.to_pos)
        board.place_piece(move.from_pos, moving)
        if captured: board.place_piece(move.to_pos, captured)

    return in_check

# ===== Mediator / Colleague =====
class IMediator(ABC):
    @abstractmethod
    def send_message(self, message, from_user): pass
    @abstractmethod
    def add_user(self, user): pass
    @abstractmethod
    def remove_user(self, user): pass

class IColleague(ABC):
    def __init__(self):
        self.mediator = None

    def set_mediator(self, mediator):
        self.mediator = mediator

    @abstractmethod
    def send(self, message): pass

    @abstractmethod
    def receive(self, message): pass

class Message:
    def __init__(self, sender_id, content):
        self.sender_id = sender_id
        self.content = content

class User(IColleague):
    def __init__(self, uid, name, score=1000):
        super().__init__()
        self.id = uid
        self.name = name
        self.score = score

    def send(self, message):
        if self.mediator:
            self.mediator.send_message(message, self)

```

```

def receive(self, message):
    print(f"{self.name} received: {message.content}")

class Match(IMediator):
    def __init__(self, match_id, white_player, black_player):
        self.id = match_id
        self.white_player = white_player
        self.black_player = black_player
        self.board = Board()
        self.rules = StandardChessRules()
        self.current_turn = Color.WHITE
        self.status = GameStatus.IN_PROGRESS
        self.move_history = []
        self.chat_history = []

        white_player.set_mediator(self)
        black_player.set_mediator(self)
        print(f"Match started: {white_player.name} (White) vs {black_player.name} (Black)")

    def _get_player_color(self, player):
        return Color.WHITE if player == self.white_player else Color.BLACK

    def make_move(self, from_pos, to_pos, player):
        if self.status != GameStatus.IN_PROGRESS:
            print("Game not in progress")
            return False

        player_color = self._get_player_color(player)
        if player_color != self.current_turn:
            print("Not your turn!")
            return False

        piece = self.board.get_piece(from_pos)
        if piece is None or piece.color != player_color:
            print("Invalid piece")
            return False

        move = Move(from_pos, to_pos, piece, self.board.get_piece(to_pos))
        if not self.rules.is_valid_move(move, self.board):
            print("Invalid move")
            return False

        self.board.move_piece(from_pos, to_pos)
        self.move_history.append(move)
        self.board.display()

```

```

        opponent = Color.BLACK if player_color == Color.WHITE else Color.WHITE
        if self.rules.is_checkmate(opponent, self.board):
            self._end_game(player, "CHECKMATE")
            return True
        if self.rules.is_stalemate(opponent, self.board):
            self._end_game(player, "STALEMATE")
            return True

        self.current_turn = opponent
        if self.rules.is_in_check(opponent, self.board):
            print("Opponent is in check!")
        return True

    def _end_game(self, winner, reason):
        self.status = GameStatus.COMPLETED
        print(f"Game ended. {winner.name} wins by {reason}")
        if winner:
            loser = self.black_player if winner == self.white_player else self.white_play
er
            winner.score += 30
            loser.score -= 20
            print("Scores updated")

    def quit_game(self, player):
        opponent = self.black_player if player == self.white_player else self.white_play
r
        self._end_game(opponent, "QUIT")
        player.score -= 50

    def send_message(self, message, from_user):
        self.chat_history.append(message)
        recipient = self.black_player if from_user == self.white_player else self.white_p
layer
        recipient.receive(message)

    def add_user(self, user):
        raise NotImplementedError("Cannot add more users to a chess match")

    def remove_user(self, user):
        self.quit_game(user)

class MatchingStrategy(ABC):
    @abstractmethod
    def find_match(self, user, waiting_users): pass

```

```
class ScoreBasedMatching(MatchingStrategy):
    def __init__(self, tolerance=100):
        self.tolerance = tolerance

    def find_match(self, user, waiting_users):
        best_match = None
        best_diff = float('inf')
        for candidate in waiting_users:
            if candidate.id == user.id:
                continue
            diff = abs(candidate.score - user.score)
            if diff <= self.tolerance and diff < best_diff:
                best_diff = diff
                best_match = candidate
        return best_match

class GameManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
        return cls._instance

    def __init__(self):
        if self._initialized:
            return
        self._initialized = True
        self.active_matches = {}
        self.waiting_users = []
        self.match_counter = 0
        self.matching_strategy = ScoreBasedMatching(100)

    def request_match(self, user):
        print(f"{user.name} is looking for a match")
        opponent = self.matching_strategy.find_match(user, self.waiting_users)

        if opponent is None:
            self.waiting_users.append(user)
            print(f"{user.name} added to waiting list")
            return None

        self.waiting_users.remove(opponent)
        self.match_counter += 1
        match_id = f"MATCH_{self.match_counter}"
        match = Match(match_id, user, opponent)
```

```

        self.active_matches[match_id] = match
        print(f"Match found: {match_id}")
        return match

    def display_active_matches(self):
        print(f"Active matches: {len(self.active_matches)}")
        print(f"Waiting users: {len(self.waiting_users)}")

# Demo
if __name__ == "__main__":
    # Scholar's mate
    aditya = User("D1", "Aditya", 1000)
    rohit = User("D2", "Rohit", 1000)
    match = Match("DEMO", aditya, rohit)

    match.make_move(Position(6,4), Position(4,4), aditya)    # e2-e4
    match.make_move(Position(1,4), Position(3,4), rohit)    # e7-e5
    match.make_move(Position(7,5), Position(4,2), aditya)    # Bf1-c4
    match.make_move(Position(0,1), Position(2,2), rohit)     # Nb8-c6
    match.make_move(Position(7,3), Position(3,7), aditya)    # Qd1-h5
    match.make_move(Position(0,6), Position(2,5), rohit)     # Ng8-f6
    match.make_move(Position(3,7), Position(1,5), aditya)    # Qh5xf7#

    match.send_message(Message(aditya.id, "Good game!"), aditya)
    match.send_message(Message(rohit.id, "That was quick!"), rohit)

    # Matchmaking
    print("\n==== MATCHMAKING =====")
    gm = GameManager()
    s = User("U1", "Sourav", 1000)
    m = User("U2", "Manish", 1000)
    a = User("U3", "Abhishek", 1000)
    gm.request_match(s)
    gm.request_match(m)
    gm.request_match(a)
    gm.display_active_matches()

```

8. Key Design Decisions

Decision	Rationale
Singleton for GameManager	Single entry point, orchestrates all matches

Decision	Rationale
Mediator for Match	Handles chat between 2 players without direct coupling
Strategy for Matchmaking	Enables swapping algorithms (score-based, ELO-based, etc.)
Abstract Piece + Concrete subclasses	Each piece has different move rules — polymorphism
PieceFactory	Centralizes piece creation, hides concrete class selection
Board as "dumb" object	Board only manages pieces; rules are separate (SRP)
Map<Position, Piece>	O(1) lookup for pieces on board
ChessRules abstract	Extensible for custom rules (e.g., Atomic Chess)
hasMoved flag in Piece	Needed for pawn double-move and castling
Move object	Encapsulates from/to/piece/captured for history & undo

9. Scholar's Mate Sequence (4-move checkmate)

text

1. White: e2-e4 (Pawn forward 2)
Black: e7-e5 (Pawn forward 2)
2. White: Bf1-c4 (Bishop develops, targeting f7)
Black: Nb8-c6 (Knight develops)
3. White: Qd1-h5 (Queen attacks f7 & h7)
Black: Ng8-f6 (Knight defends h7 – but this is a mistake)
4. White: Qh5xf7# (Queen captures pawn f7 → CHECKMATE)

Black's king cannot escape (surrounded by own pieces) and can't capture the queen (defended by the bishop on c4).

10. Interview Tips

When asked this LLD question:

1. **Start with requirements** — functional + non-functional
2. **Identify key entities:** Game, Board, Piece, User, Match, Move, Position
3. **Explain patterns used:**
 - **Singleton** → GameManager
 - **Strategy** → Matchmaking algorithm
 - **Mediator** → Chat
 - **Factory** → Piece creation
 - **Template Method** → Piece.getPossibleMoves
4. **Focus on extensibility** — ChessRules can be subclassed
5. **Mention edge cases** — Checkmate, stalemate, en passant, castling
6. **Explain complexity** — $O(1)$ piece lookups via Map; $O(64)$ worst-case board scans
7. **Discuss scaling** — Multiple concurrent matches via `Map<String, Match>`

11. Key Takeaways

1. **Top-down design** for board games — start with orchestrator, drill down
2. **Mediator pattern** naturally fits chat rooms within matches
3. **Strategy pattern** decouples matchmaking logic
4. **Piece polymorphism** elegantly handles different move rules
5. **Board = dumb storage**; Rules = smart validator (SRP)
6. **GameManager = Façade** that delegates to Match
7. **Concise flow:** `requestMatch` → `findMatch` → `createMatch` → `makeMove` → `endGame`
8. Real-world: add frontend → production-ready chess.com clone
9. Interview variants: only board, only matchmaking, only chat, or the full design



हां जी। तो करें स्टार्ट। वेलकम ...
TXT 109.3KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python

also

Visitor Design Pattern - Complete Notes

1. Introduction & Purpose

The **Visitor Design Pattern** is a behavioral design pattern that:

- Allows adding new operations to existing class hierarchies without modifying them
- Separates an algorithm (operation) from the object structure it operates on
- Uses **double dispatch** to determine which method to call
- Promotes the **Open/Closed Principle** by allowing new operations without changing existing classes

Standard Definition

"Allows you to add new operations to existing object structures without modifying those structures. It separates the operation from the object it operates on."

2. The Problem It Solves

Core Problem: Adding Operations to Class Hierarchies

Consider a class `A` with methods `m1` and `m2`. New requirements keep arriving:

text

Requirement 1: Add `m3` → Modify class `A`
Requirement 2: Add `m4` → Modify class `A` again
Requirement 3: Add `m5` → Modify class `A` again
...

Problems with this approach:

Principle Violated	Why
Open/Closed Principle (OCP)	Class is repeatedly modified instead of extended

Principle Violated	Why
Single Responsibility Principle (SRP)	Class handles multiple unrelated responsibilities
Maintainability	Class becomes bloated and complex
Testability	Hard to unit test individual methods
Extensibility	Every new operation forces class changes

Real-World Example: Document Elements

Suppose you have a document application that handles different document types:

- Text files
- Image files
- Video files

Each document type can have operations like:

- Calculate size
- Compress
- Scan for viruses
- Generate preview
- Extract metadata
- ... (operations keep growing)

Without Visitor Pattern (Bad Design):

```
java
```

```
abstract class DocumentElement {  
    public abstract void calculateSize();  
    public abstract void compress();  
    public abstract void scanForVirus();  
    // Every new operation forces us to modify every subclass!  
}
```

```
class TextFile extends DocumentElement {  
    public void calculateSize() { /* logic */ }  
    public void compress() { /* logic */ }
```

```
public void scanForVirus() { /* Logic */ }  
}  
  
class ImageFile extends DocumentElement {  
    public void calculateSize() { /* Logic */ }  
    public void compress() { /* Logic */ }  
    public void scanForVirus() { /* Logic */ }  
}  
  
class VideoFile extends DocumentElement {  
    public void calculateSize() { /* Logic */ }  
    public void compress() { /* Logic */ }  
    public void scanForVirus() { /* Logic */ }  
}
```

Issues:

- Every new operation (e.g., `generatePreview()`) requires changes in **all** classes
- Logic for the same operation is scattered across many classes
- Class hierarchy becomes huge and unmaintainable

3. The Visitor Solution

Key Insight: Extract What Changes

Design patterns teach us to "encapsulate what varies."

- **What varies (frequently):** Operations on documents (size, compress, scan, preview, etc.)
- **What stays stable (rarely):** Document types (Text, Image, Video)

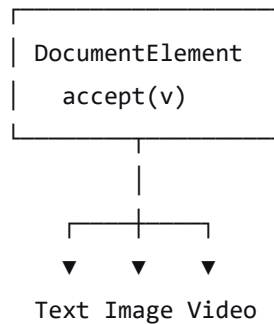
Note: We make an assumption that document types don't change frequently. If they did, Visitor might not be the best fit.

How Visitor Works

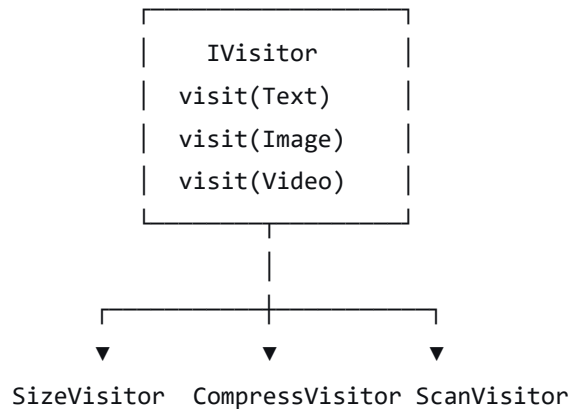
1. **Move all operations out** of the document classes into separate "Visitor" classes
2. **Document classes keep only one method** — `accept(Visitor v)`
3. **Each Visitor class** handles a specific operation for all document types
4. When a new operation is needed → **create a new Visitor** (no changes to documents!)

text

Document hierarchy (stable)



Visitor hierarchy (grows freely)



4. Double Dispatch - The Core Mechanism

Visitor pattern relies on **double dispatch** — a concept often considered advanced but actually simple.

What is Dispatch?

Dispatch = The process of determining which method to call.

- **Single Dispatch:** Only one reference (the object) decides which method is called
- **Double Dispatch:** Two references together decide which method is called

How Double Dispatch Works Here

Look at client code:

java

```

DocumentElement file = new TextFile();
file.accept(new SizeCalculatorVisitor());
  
```

Step 1: First Dispatch

- `file.accept(...)` — Here, `file` (a `TextFile`) determines which `accept()` method is called
- **Result:** `TextFile.accept()` is called

Step 2: Second Dispatch

- Inside `TextFile.accept()` :

java

```
public void accept(IVisitor v) {
    v.visit(this); // 'this' is a TextFile reference
}
```

- Now `v` (`SizeCalculatorVisitor`) + `this` (`TextFile`) together determine which `visit()` method is called
- Result: `SizeCalculatorVisitor.visit(TextFile)` is called

Why is This "Double"?

Two references cooperate:

1. **Document reference** (`file / this`) → identifies which `accept()` and passes itself
2. **Visitor reference** (`v`) → identifies which `visit()` overload gets called

Single dispatch (typical): `obj.method()` → only `obj` decides

Double dispatch (Visitor): `obj.accept(visitor)` → both `obj` AND `visitor` decide

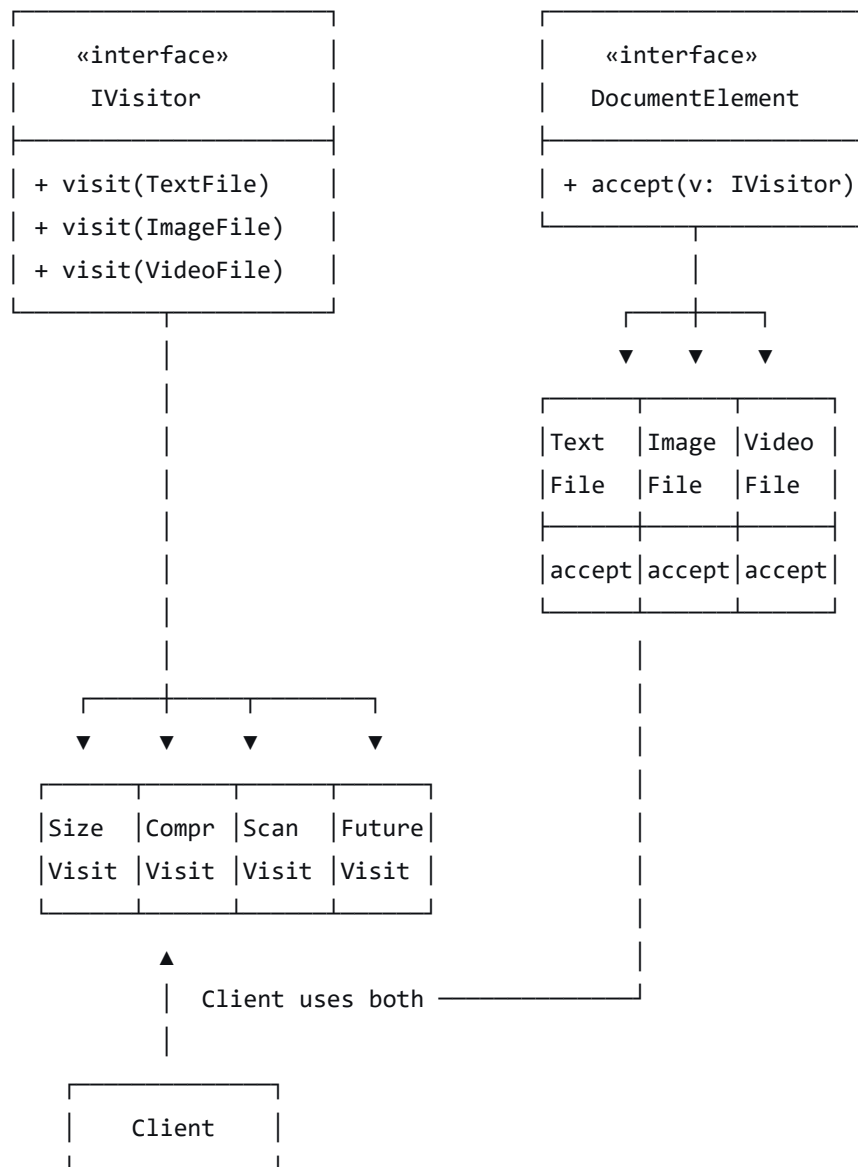
Visualizing Double Dispatch

text

```
file.accept(sizeVisitor)
|
| [First Dispatch] - file identifies itself
▼
TextFile.accept(sizeVisitor)
|
| [Second Dispatch] - sizeVisitor.visit(this)
▼
sizeVisitor.visit(TextFile) ← Correct method called!
```

5. UML Class Diagram

text



6. Java Implementation

6.1 Visitor Interface

java

```

// IVisitor - abstract parent class for all visitors
public interface IVisitor {
    // One visit method per concrete element type (method overloading)
    void visit(TextFile textFile);
    void visit(ImageFile imageFile);
    void visit(VideoFile videoFile);
}
  
```

6.2 Document Element Hierarchy

```
java

// Abstract Document Element
public abstract class DocumentElement {
    protected String name;

    public DocumentElement(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // Only ONE method – accept. Never changes.
    public abstract void accept(IVisitor visitor);
}
```

6.3 Concrete Document Elements

```
java

// TextFile
public class TextFile extends DocumentElement {
    private String content;

    public TextFile(String name, String content) {
        super(name);
        this.content = content;
    }

    public String getContent() {
        return content;
    }

    @Override
    public void accept(IVisitor visitor) {
        visitor.visit(this); // 'this' is a TextFile
    }
}

// ImageFile
public class ImageFile extends DocumentElement {
    private int width;
    private int height;
```

```

    public ImageFile(String name, int width, int height) {
        super(name);
        this.width = width;
        this.height = height;
    }

    public int getWidth() { return width; }
    public int getHeight() { return height; }

    @Override
    public void accept(IVisitor visitor) {
        visitor.visit(this);    // 'this' is an ImageFile
    }
}

// VideoFile
public class VideoFile extends DocumentElement {
    private int durationSeconds;

    public VideoFile(String name, int durationSeconds) {
        super(name);
        this.durationSeconds = durationSeconds;
    }

    public int getDurationSeconds() {
        return durationSeconds;
    }

    @Override
    public void accept(IVisitor visitor) {
        visitor.visit(this);    // 'this' is a VideoFile
    }
}

```

6.4 Concrete Visitors

java

```

// Visitor 1: Calculate Size
public class SizeCalculatorVisitor implements IVisitor {

    @Override
    public void visit(TextFile textFile) {
        int size = textFile.getContent().length();
        System.out.println("Calculating size of Text File: " + size + " bytes");
    }
}

```

```
@Override
public void visit(ImageFile imageFile) {
    int size = imageFile.getWidth() * imageFile.getHeight() * 4; // RGBA
    System.out.println("Calculating size of Image File: " + size + " bytes");
}

@Override
public void visit(VideoFile videoFile) {
    int size = videoFile.getDurationSeconds() * 1024 * 1024; // 1MB/sec
    System.out.println("Calculating size of Video File: " + size + " bytes");
}
}

// Visitor 2: Compress
public class CompressVisitor implements IVisitor {

    @Override
    public void visit(TextFile textFile) {
        System.out.println("Compressing Text File using ZIP: " + textFile.getName());
    }

    @Override
    public void visit(ImageFile imageFile) {
        System.out.println("Compressing Image File using JPEG: " + imageFile.getName());
    }

    @Override
    public void visit(VideoFile videoFile) {
        System.out.println("Compressing Video File using H.264: " + videoFile.getName());
    }
}

// Visitor 3: Scan for Virus
public class ScanVirusVisitor implements IVisitor {

    @Override
    public void visit(TextFile textFile) {
        System.out.println("Scanning Text File for viruses: " + textFile.getName());
    }

    @Override
    public void visit(ImageFile imageFile) {
        System.out.println("Scanning Image File for viruses: " + imageFile.getName());
    }

    @Override
    public void visit(VideoFile videoFile) {
```

```

        System.out.println("Scanning Video File for viruses: " + videoFile.getName());
    }
}

```

6.5 Client Code

java

```

public class Main {
    public static void main(String[] args) {
        // Create document elements
        DocumentElement text = new TextFile("notes.txt", "Hello World!");
        DocumentElement image = new ImageFile("photo.jpg", 1920, 1080);
        DocumentElement video = new VideoFile("movie.mp4", 120);

        // Create visitors
        IVisitor sizeVisitor = new SizeCalculatorVisitor();
        IVisitor compressVisitor = new CompressVisitor();
        IVisitor scanVisitor = new ScanVirusVisitor();

        // Apply different operations to same element
        System.out.println("--- Size Operations ---");
        text.accept(sizeVisitor);
        image.accept(sizeVisitor);
        video.accept(sizeVisitor);

        System.out.println("\n--- Compression Operations ---");
        text.accept(compressVisitor);
        image.accept(compressVisitor);
        video.accept(compressVisitor);

        System.out.println("\n--- Virus Scan Operations ---");
        text.accept(scanVisitor);
        image.accept(scanVisitor);
        video.accept(scanVisitor);
    }
}

```

Expected Output

text

```

--- Size Operations ---
Calculating size of Text File: 12 bytes
Calculating size of Image File: 8294400 bytes

```

Calculating size of Video File: 125829120 bytes

--- Compression Operations ---

Compressing Text File using ZIP: notes.txt

Compressing Image File using JPEG: photo.jpg

Compressing Video File using MOV: movie.mp4

--- Virus Scan Operations ---

Scanning Text File for viruses: notes.txt

Scanning Image File for viruses: photo.jpg

Scanning Video File for viruses: movie.mp4

7. Python Implementation

python

```
from abc import ABC, abstractmethod
```

```
# ===== Document Element Hierarchy =====
```

```
class DocumentElement(ABC):
```

```
    """Abstract element with only one operation: accept"""
```

```
    def __init__(self, name: str):
```

```
        self.name = name
```

```
    def get_name(self) -> str:
```

```
        return self.name
```

```
    @abstractmethod
```

```
    def accept(self, visitor: 'IVisitor') -> None:
```

```
        pass
```

```
class TextFile(DocumentElement):
```

```
    def __init__(self, name: str, content: str):
```

```
        super().__init__(name)
```

```
        self.content = content
```

```
    def get_content(self) -> str:
```

```
        return self.content
```

```
    def accept(self, visitor: 'IVisitor') -> None:
```

```
        visitor.visit_text_file(self) # pass 'self' as TextFile
```

```

class ImageFile(DocumentElement):
    def __init__(self, name: str, width: int, height: int):
        super().__init__(name)
        self.width = width
        self.height = height

    def get_width(self) -> int:
        return self.width

    def get_height(self) -> int:
        return self.height

    def accept(self, visitor: 'IVisitor') -> None:
        visitor.visit_image_file(self)

class VideoFile(DocumentElement):
    def __init__(self, name: str, duration_seconds: int):
        super().__init__(name)
        self.duration_seconds = duration_seconds

    def get_duration(self) -> int:
        return self.duration_seconds

    def accept(self, visitor: 'IVisitor') -> None:
        visitor.visit_video_file(self)

# ===== Visitor Hierarchy =====

class IVisitor(ABC):
    """Abstract visitor with one visit method per element type"""

    @abstractmethod
    def visit_text_file(self, text_file: TextFile) -> None:
        pass

    @abstractmethod
    def visit_image_file(self, image_file: ImageFile) -> None:
        pass

    @abstractmethod
    def visit_video_file(self, video_file: VideoFile) -> None:
        pass

```

```

class SizeCalculatorVisitor(IVisitor):
    def visit_text_file(self, text_file):
        size = len(text_file.get_content())
        print(f"Calculating size of Text File: {size} bytes")

    def visit_image_file(self, image_file):
        size = image_file.get_width() * image_file.get_height() * 4
        print(f"Calculating size of Image File: {size} bytes")

    def visit_video_file(self, video_file):
        size = video_file.get_duration() * 1024 * 1024
        print(f"Calculating size of Video File: {size} bytes")

class CompressVisitor(IVisitor):
    def visit_text_file(self, text_file):
        print(f"Compressing Text File using ZIP: {text_file.get_name()}")

    def visit_image_file(self, image_file):
        print(f"Compressing Image File using JPEG: {image_file.get_name()}")

    def visit_video_file(self, video_file):
        print(f"Compressing Video File using H.264: {video_file.get_name()}")

class ScanVirusVisitor(IVisitor):
    def visit_text_file(self, text_file):
        print(f"Scanning Text File for viruses: {text_file.get_name()}")

    def visit_image_file(self, image_file):
        print(f"Scanning Image File for viruses: {image_file.get_name()}")

    def visit_video_file(self, video_file):
        print(f"Scanning Video File for viruses: {video_file.get_name()}")

# ===== Client =====

def main():
    # Create documents
    text = TextFile("notes.txt", "Hello World!")
    image = ImageFile("photo.jpg", 1920, 1080)
    video = VideoFile("movie.mp4", 120)

    # Create visitors
    size_visitor = SizeCalculatorVisitor()
    compress_visitor = CompressVisitor()
    scan_visitor = ScanVirusVisitor()

```

```

# Apply different operations
print("--- Size Operations ---")
text.accept(size_visitor)
image.accept(size_visitor)
video.accept(size_visitor)

print("\n--- Compression Operations ---")
text.accept(compress_visitor)
image.accept(compress_visitor)
video.accept(compress_visitor)

print("\n--- Virus Scan Operations ---")
text.accept(scan_visitor)
image.accept(scan_visitor)
video.accept(scan_visitor)

if __name__ == "__main__":
    main()

```

Python Alternative: Using `functools.singledispatch` (Function-Based Visitor)

Python doesn't have native method overloading, so a function-based visitor using `@singledispatch` is a common alternative:

python

```

from functools import singledispatch

@singledispatch
def calculate_size(doc):
    raise NotImplementedError(f"No size calculation for {type(doc)}")

@calculate_size.register
def _(doc: TextFile):
    print(f"Text size: {len(doc.get_content())} bytes")

@calculate_size.register
def _(doc: ImageFile):
    print(f"Image size: {doc.get_width() * doc.get_height() * 4} bytes")

```

```
@calculate_size.register
def _(doc: VideoFile):
    print(f"Video size: {doc.get_duration() * 1024 * 1024} bytes")

# Usage:
calculate_size(TextFile("a.txt", "hello"))
calculate_size(ImageFile("b.jpg", 100, 100))
calculate_size(VideoFile("c.mp4", 60))
```

8. Visitor vs Strategy Pattern

A very common interview question — many confuse these because both look similar at first glance. **But they solve completely different problems.**

Core Differences

Aspect	Strategy Pattern	Visitor Pattern
Problem Solved	How to do ONE thing in MANY ways	How to add MANY operations to a class
What Varies	Implementation/algorithm of a behavior	Number of operations on an object
Number of Operations	Fixed	Grows over time
Number of Variants	Grows (new algorithms)	Fixed per operation
Focus	Interchangeable algorithms	Adding operations without modification
Uses Double Dispatch	No	Yes
Modifies Object Class?	No, but object holds strategy reference	No, object has one <code>accept</code> method

Concrete Comparison: Robot Example

Strategy Perspective:

- Robot has **fixed behaviors**: `fly()` , `talk()` , `walk()`
- But each behavior has **multiple ways**: `FlyWithWings` , `FlyWithJet` , `NoFly` , etc.
- If you want new behavior `swim()` → you must modify Robot class (breaks OCP)
- Strategy is best when behavior SET is fixed, but IMPLEMENTATIONS vary

Visitor Perspective:

- Robot needs to support **ever-growing operations**: `fly` , `talk` , `walk` , `swim` , `dance` ...
- Each operation has **one implementation** per element type
- Adding `swim()` → just create `SwimVisitor` (no Robot changes!)
- Visitor is best when OPERATIONS grow, but IMPLEMENTATION VARIANTS are minimal

Code Contrast

Strategy (Robot with fixed behaviors):

```
java

class Robot {
    private IFlyStrategy flyStrategy;
    private ITalkStrategy talkStrategy;
    private IWalkStrategy walkStrategy;

    public void fly() { flyStrategy.fly(); }
    public void talk() { talkStrategy.talk(); }
    public void walk() { walkStrategy.walk(); }

    // New behavior? Must add here! Breaks OCP.
}

class FlyWithWings implements IFlyStrategy { /*...*/ }
class FlyWithJet implements IFlyStrategy { /*...*/ }
```

Visitor (Robot with growing operations):

```
java

class Robot {
    public void accept(IRobotVisitor visitor) {
        visitor.visit(this);
    }

    // NEVER changes! No matter how many operations!
}
```

```
interface IRobotVisitor {  
    void visit(Robot robot);  
}  
  
class FlyVisitor implements IRobotVisitor {  
    public void visit(Robot robot) { /* fly logic */ }  
}  
class TalkVisitor implements IRobotVisitor { /*...*/ }  
class SwimVisitor implements IRobotVisitor { /*...*/ } // Add freely!
```

Decision Guide

Use Strategy when:

- Set of behaviors is fixed
- Implementation of each behavior varies
- Example: Payment methods (Credit, Debit, UPI) — always 3, but implementations differ

Use Visitor when:

- Set of operations grows over time
- Each operation has one clear implementation per element type
- Example: Document operations (compress, scan, preview, export...) — grows indefinitely

Trade-off:

- If BOTH behaviors AND implementations change → use Strategy (simpler), accept minor OCP violation
- Visitor is more complex but more extensible for growing operations

9. Advantages & Disadvantages

Advantages

- ✓ **Open/Closed Principle:** Add new operations without modifying existing classes
- ✓ **Single Responsibility:** Each operation lives in its own class

- ✓ **Separation of Concerns:** Algorithms separated from object structure
- ✓ **Accumulation of State:** Visitor can accumulate data across multiple elements
- ✓ **Clean Code:** Operations are grouped logically by purpose

Disadvantages

- ✗ **Hard to Add Element Types:** Adding a new element requires changes to all visitors
- ✗ **Breaks Encapsulation:** Visitors often need access to element internals
- ✗ **Complex:** Double dispatch can confuse beginners
- ✗ **Overkill for Simple Cases:** Not needed for one-off operations

When to Use

Use Visitor when:

1. Object structure is stable (elements rarely change)
2. Many distinct operations need to be performed
3. Operations need to accumulate state across elements
4. You want to group related operations in one class

Avoid Visitor when:

1. Element types change frequently
2. Few operations are needed
3. Operations don't share state
4. You can't afford to modify all visitors when adding an element

10. Real-World Use Cases

Use Case	Why Visitor?
Compilers (AST traversal)	Tree of AST nodes; many operations (type check, code gen, optimize)
Game Rendering Engines	3D scene graph nodes; many rendering passes (shadow, lighting, effects)

Use Case	Why Visitor?
Document Processing	Different document types with many operations (export, scan, preview)
XML/HTML Parsing	DOM tree with varied operations (validation, transformation, serialization)
UI Toolkits	Widgets with growing operations (render, event handling, layout)
Data Serialization	Different data types with many formats (JSON, XML, YAML)
Insurance/Risk Analysis	Different asset types with many assessment operations

11. Key Takeaways

1. **Visitor = Add operations without modifying element classes**
2. **Double dispatch:** Two references together decide the method to call
3. `accept(visitor)` **on element** + `visitor.visit(element)` = The core mechanism
4. **Separate hierarchy for visitors** — one visitor per operation
5. **Use when operations grow**, not when element types grow
6. **Different from Strategy:** Strategy = how to do ONE thing; Visitor = add MANY operations
7. **Real-world: Compilers, game engines, document processors**
8. **Trade-off:** More classes and complexity for extensibility

12. Interview Tips

When asked about Visitor pattern:

1. **State the problem:** "Adding operations to a class hierarchy without modifying it"
2. **Explain double dispatch:** "Both object and visitor participate in method selection"
3. **Show the core code:** `accept(visitor) → visitor.visit(this)`

4. **Highlight OCP benefit:** "New operations = new visitor, no changes to elements"
5. **Compare with Strategy:** Critical — many interviewers test this
6. **Mention trade-offs:** "Hard to add new element types"
7. **Real-world examples:** Compilers (AST), game engines, document processing

Key phrase: "Visitor lets you add new operations to existing class hierarchies without modifying them, using double dispatch to select the right method at runtime."

13. Complete Flow Summary

text

1. Client has a DocumentElement (e.g., TextFile)
2. Client creates a Visitor (e.g., SizeCalculatorVisitor)
3. Client calls: `textFile.accept(sizeVisitor)`
4. `TextFile.accept()` calls: `sizeVisitor.visit(this)`
5. Because 'this' is a TextFile, `visit(TextFile)` is called
6. The specific operation logic for TextFile executes
7. Repeat for other elements/visitors freely – no changes needed

Adding a new operation (e.g., generatePreview):

- Create new PreviewVisitor implements IVisitor
- Implement `visit(TextFile)`, `visit(ImageFile)`, `visit(VideoFile)`
- Done! No changes to DocumentElement classes!



हां जी तो करें स्टार्ट। वेलकम बै...
TXT 92.78KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Memento Design Pattern - Complete Notes

1. Introduction & Purpose

The **Memento Design Pattern** is a behavioral design pattern that:

- **Captures and externalizes an object's internal state** without violating encapsulation
- Allows the object to be **restored to this state later**

- Provides **undo/rollback capabilities**
- Uses **three key components**: Originator, Memento, and Caretaker

Standard Definition

"Provides the ability to take a snapshot of an object at various points in time and provides undo capabilities to a previous state."

2. The Problem It Solves

Core Problem: State Rollback

Consider an object `OB1` whose state keeps changing:

text

`OB1 (State 1) → OB1' (State 2) → OB1'' (State 3) → ...`

At any point, you might want to **rollback** to a previous state because:

- Current state is **unstable** (errors, wrong output)
- User wants **undo** functionality
- A **failure occurred** and you need consistency

Real-World Example: Database Transactions

Consider a database row:

text

ID: 1001, Name: Aditya, Role: X

You want to update it to:

text

ID: 102, Name: Abhay, Role: Y

If the transaction fails halfway:

text

ID: 102, Name: Aditya, Role: X ← INCONSISTENT!

The row is in an **inconsistent state** — ID changed but Name didn't. We need **Atomicity** — either all changes succeed or none do.

Solution: Take a snapshot before changes. If failure occurs, **rollback** to the snapshot.

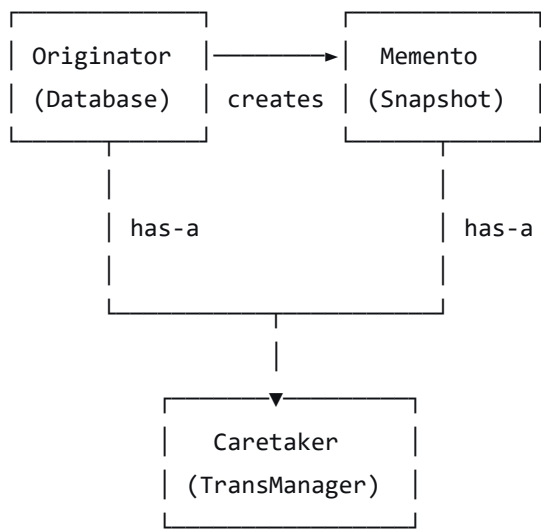
This is exactly what **Memento** does.

3. The Three Key Components

Component	Role	Example
Originator	The object whose state changes	Database
Memento	Snapshot of the originator's state	DatabaseMemento
Caretaker	Manages/saves mementos	TransactionManager

Relationships

text



Flow:

1. **Originator** creates a **Memento** containing its current state

2. **Caretaker** stores the Memento (doesn't peek inside)
3. Later, Caretaker gives Memento back to Originator
4. Originator restores itself from Memento

4. Detailed Design

4.1 The Originator (Database)

The Originator:

- Holds the actual state (e.g., `Map<String, String>` records)
- Has methods to modify state (`insert` , `update` , `remove`)
- Has `createMemento()` — creates snapshot
- Has `restoreFromMemento(m)` — restores from snapshot

Key insight: The Originator knows what state needs to be saved. It delegates to Memento.

4.2 The Memento (Snapshot)

The Memento:

- Holds a **copy** of the Originator's state
- Has `getState()` — returns saved state
- Has `setState(state)` — saves state (or use constructor)
- **Doesn't expose internals to Caretaker**

Key insight: Only the Originator should know the Memento's internal structure. Caretaker just holds it like a black box.

4.3 The Caretaker (Transaction Manager)

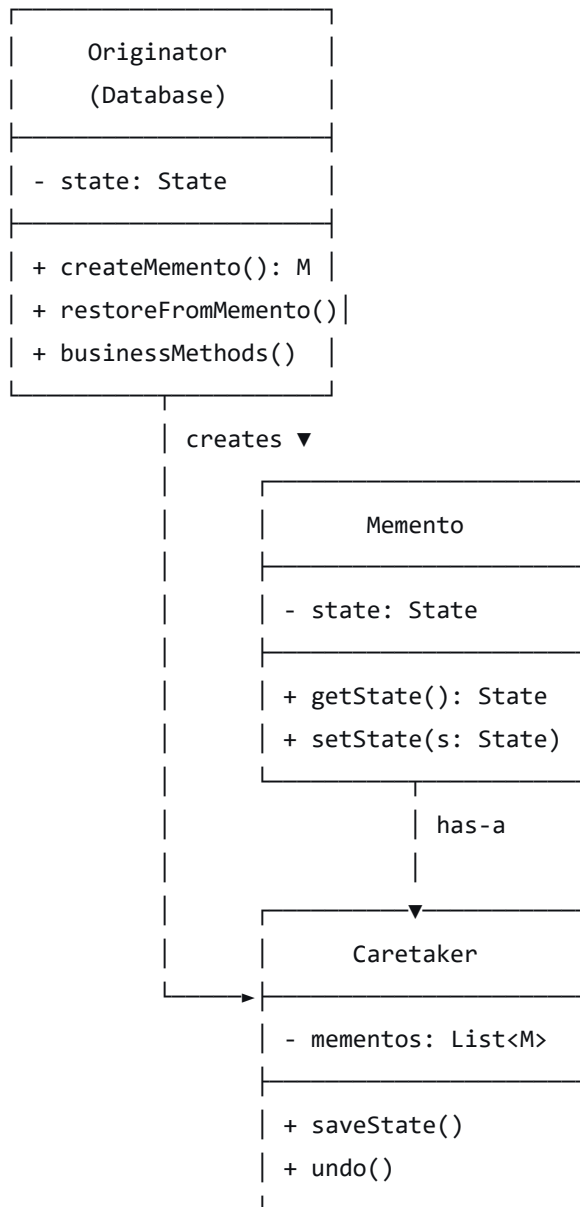
The Caretaker:

- Stores one or more Mementos
- Never inspects or modifies the Memento's content
- Triggers save/restore operations on the Originator

Key insight: Caretaker doesn't know what state looks like. It just manages the snapshot lifecycle.

5. UML Class Diagram

text



6. Java Implementation

6.1 State (Optional Wrapper)

```
java
```

```
// Optional – could be a simple Map
public class State {
    private final Map<String, String> data;

    public State(Map<String, String> data) {
        this.data = new HashMap<>(data); // defensive copy
    }

    public Map<String, String> getData() {
        return data;
    }
}
```

6.2 Memento (Snapshot)

```
java
```

```
public class DatabaseMemento {
    private final Map<String, String> data;

    public DatabaseMemento(Map<String, String> data) {
        // Store a copy – this is the snapshot
        this.data = new HashMap<>(data);
    }

    public Map<String, String> getState() {
        // Return a copy to prevent external modification
        return new HashMap<>(data);
    }
}
```

6.3 Originator (Database)

```
java
```

```
public class Database {
    private Map<String, String> records = new HashMap<>();

    // ===== Business operations =====

    public void insert(String key, String value) {
        records.put(key, value);
        System.out.println("Inserted: " + key + " = " + value);
    }
}
```

```

    }

    public void update(String key, String value) {
        if (records.containsKey(key)) {
            records.put(key, value);
            System.out.println("Updated: " + key + " = " + value);
        }
    }

    public void remove(String key) {
        records.remove(key);
        System.out.println("Removed: " + key);
    }

    // ===== Memento operations =====

    public DatabaseMemento createMemento() {
        System.out.println("Creating database snapshot...");
        return new DatabaseMemento(records);
    }

    public void restoreFromMemento(DatabaseMemento memento) {
        this.records = memento.getState();
        System.out.println("Database restored from snapshot.");
    }

    public void displayRecords() {
        System.out.println("Current Records: " + records);
    }
}

```

6.4 Caretaker (Transaction Manager)

java

```

public class TransactionManager {
    private DatabaseMemento backup; // single memento for this example

    public void beginTransaction(Database db) {
        if (backup != null) {
            backup = null; // discard previous
        }
        backup = db.createMemento();
        System.out.println("Transaction started.");
    }

    public void commitTransaction() {

```

```

        if (backup != null) {
            backup = null;
            System.out.println("Transaction committed successfully.");
        }
    }

    public void rollbackTransaction(Database db) {
        if (backup != null) {
            db.restoreFromMemento(backup);
            backup = null;
            System.out.println("Transaction rolled back.");
        } else {
            System.out.println("No backup available. Cannot rollback.");
        }
    }
}

```

6.5 Caretaker with History (Multiple Mementos)

java

```

// Caretaker that supports undo (multiple snapshots)
public class HistoryManager {
    private final Stack<DatabaseMemento> history = new Stack<>();
    private final Database db;

    public HistoryManager(Database db) {
        this.db = db;
    }

    public void save() {
        history.push(db.createMemento());
        System.out.println("Snapshot saved. Total: " + history.size());
    }

    public void undo() {
        if (history.isEmpty()) {
            System.out.println("Nothing to undo!");
            return;
        }
        DatabaseMemento m = history.pop();
        db.restoreFromMemento(m);
        System.out.println("Undo performed.");
    }
}

```

6.6 Client Code

java

```
public class Main {
    public static void main(String[] args) {
        Database db = new Database();
        TransactionManager tm = new TransactionManager();

        // Transaction 1: Successful
        tm.beginTransaction(db);
        db.insert("user1", "Aditya");
        db.insert("user2", "Rohit");
        tm.commitTransaction();
        db.displayRecords();

        // Transaction 2: Failed - simulate error
        tm.beginTransaction(db);
        db.insert("user3", "Sourav");
        db.insert("user4", "Manish");
        db.displayRecords(); // Shows 4 records

        System.out.println("\n--- Simulating ERROR ---");
        tm.rollbackTransaction(db);
        db.displayRecords(); // Should show only 2 records
    }
}
```

Expected Output

text

```
Transaction started.
Creating database snapshot...
Inserted: user1 = Aditya
Inserted: user2 = Rohit
Transaction committed successfully.
Current Records: {user1=Aditya, user2=Rohit}

Transaction started.
Creating database snapshot...
Inserted: user3 = Sourav
Inserted: user4 = Manish
Current Records: {user1=Aditya, user2=Rohit, user3=Sourav, user4=Manish}

--- Simulating ERROR ---
Database restored from snapshot.
```

Database restored from snapshot.

Transaction rolled back.

Current Records: {user1=Aditya, user2=Rohit}

7. Python Implementation

python

```
import copy
from typing import Dict, List

# ===== Memento =====

class DatabaseMemento:
    """Snapshot of the database state"""

    def __init__(self, data: Dict[str, str]):
        # Store a deep copy to ensure isolation
        self._data = copy.deepcopy(data)

    def get_state(self) -> Dict[str, str]:
        # Return a copy to prevent external mutation
        return copy.deepcopy(self._data)

# ===== Originator =====

class Database:
    """The Originator – holds state that changes"""

    def __init__(self):
        self._records: Dict[str, str] = {}

    # ===== Business operations =====

    def insert(self, key: str, value: str) -> None:
        self._records[key] = value
        print(f"Inserted: {key} = {value}")

    def update(self, key: str, value: str) -> None:
        if key in self._records:
            self._records[key] = value
            print(f"Updated: {key} = {value}")
```

```

def remove(self, key: str) -> None:
    self._records.pop(key, None)
    print(f"Removed: {key}")

# ===== Memento operations =====

def create_memento(self) -> DatabaseMemento:
    print("Creating database snapshot...")
    return DatabaseMemento(self._records)

def restore_from_memento(self, memento: DatabaseMemento) -> None:
    self._records = memento.get_state()
    print("Database restored from snapshot.")

def display_records(self) -> None:
    print(f"Current Records: {self._records}")

# ===== Caretaker =====

class TransactionManager:
    """Manages mementos – single backup for transaction rollback"""

    def __init__(self):
        self._backup: DatabaseMemento = None

    def begin_transaction(self, db: Database) -> None:
        if self._backup is not None:
            self._backup = None
        self._backup = db.create_memento()
        print("Transaction started.")

    def commit_transaction(self) -> None:
        if self._backup is not None:
            self._backup = None
            print("Transaction committed successfully.")

    def rollback_transaction(self, db: Database) -> None:
        if self._backup is not None:
            db.restore_from_memento(self._backup)
            self._backup = None
            print("Transaction rolled back.")
        else:
            print("No backup available. Cannot rollback.")

# ===== Caretaker with History (undo) =====

```

```

class HistoryManager:
    """Caretaker that supports multiple undos"""

    def __init__(self, db: Database):
        self._db = db
        self._history: List[DatabaseMemento] = []

    def save(self) -> None:
        self._history.append(self._db.create_memento())
        print(f"Snapshot saved. Total: {len(self._history)}")

    def undo(self) -> None:
        if not self._history:
            print("Nothing to undo!")
            return
        m = self._history.pop()
        self._db.restore_from_memento(m)
        print("Undo performed.")

# ===== Client =====

if __name__ == "__main__":
    db = Database()
    tm = TransactionManager()

    # Transaction 1: Successful
    tm.begin_transaction(db)
    db.insert("user1", "Aditya")
    db.insert("user2", "Rohit")
    tm.commit_transaction()
    db.display_records()

    # Transaction 2: Failed – simulate error
    tm.begin_transaction(db)
    db.insert("user3", "Sourav")
    db.insert("user4", "Manish")
    db.display_records()

    print("\n--- Simulating ERROR ---")
    tm.rollback_transaction(db)
    db.display_records()

    # ===== Undo with history =====
    print("\n===== UNDO DEMO =====")
    db2 = Database()
    hm = HistoryManager(db2)

```

```

db2.insert("a", "1")
hm.save()
db2.insert("b", "2")
hm.save()
db2.insert("c", "3")
db2.display_records() # a, b, c

hm.undo()             # back to a, b
db2.display_records()
hm.undo()             # back to a
db2.display_records()

```

Expected Output

text

```

Transaction started.
Creating database snapshot...
Inserted: user1 = Aditya
Inserted: user2 = Rohit
Transaction committed successfully.
Current Records: {'user1': 'Aditya', 'user2': 'Rohit'}

Transaction started.
Creating database snapshot...
Inserted: user3 = Sourav
Inserted: user4 = Manish
Current Records: {'user1': 'Aditya', 'user2': 'Rohit', 'user3': 'Sourav', 'user4': 'Manish'}

--- Simulating ERROR ---
Database restored from snapshot.
Transaction rolled back.
Current Records: {'user1': 'Aditya', 'user2': 'Rohit'}

===== UNDO DEMO =====
Inserted: a = 1
Creating database snapshot...
Snapshot saved. Total: 1
Inserted: b = 2
Creating database snapshot...
Snapshot saved. Total: 2
Inserted: c = 3
Current Records: {'a': '1', 'b': '2', 'c': '3'}
Database restored from snapshot.
Undo performed.
Current Records: {'a': '1', 'b': '2'}

```

Database restored from snapshot.

Undo performed.

Current Records: {'a': '1'}

8. The Complete Flow (Database Transaction)

text

1. Client → `TransactionManager.beginTransaction(db)`
 - ↳ `TransactionManager → db.createMemento()`
 - ↳ Originator creates snapshot, returns to Caretaker
 - ↳ Caretaker stores memento in backup variable
2. Client → `db.insert(...)` / `update(...)` / `remove(...)`
 - ↳ State changes, but memento is preserved
3. Client decides:
 - |— If success: `TransactionManager.commitTransaction()`
 - | ↳ Caretaker discards memento
 - |— If failure: `TransactionManager.rollbackTransaction(db)`
 - ↳ Caretaker → `db.restoreFromMemento(backup)`
 - ↳ Originator restores state from memento

9. Encapsulation — The Critical Benefit

Why is Memento special?

Without Memento, if you wanted to save an object's state:

- You'd need **public getters** for all internal fields → **breaks encapsulation**
- Or serialize the object → **exposes internal structure**

Memento solves this by:

- Only the **Originator** knows how to create/read a Memento
- The **Caretaker** stores Mementos but **never inspects** them
- `getState()` is often **package-private** or restricted (accessible only to Originator)

Real Java example: Java's `Calendar` / `Date` and serialization mechanisms use this concept. `Object.clone()` and serialization are related.

10. Advantages & Disadvantages

Advantages

- ✓ **Encapsulation preserved** — internal state not exposed
- ✓ **Undo/rollback support** — restore to any saved point
- ✓ **Separation of concerns** — Originator, Memento, Caretaker have distinct roles
- ✓ **Multiple snapshots** — can maintain history
- ✓ **Clean code** — state saving logic centralized

Disadvantages

- ✗ **Memory overhead** — each Memento copies state; many can exhaust RAM
- ✗ **Performance cost** — deep copies are expensive
- ✗ **Complexity** — three classes needed
- ✗ **No history limit inherently** — must manage pruning manually
- ✗ **Caretaker doesn't know size** — can't enforce limits intelligently

When to Use

Use Memento when:

1. You need **undo/redo** functionality
2. You need **transaction rollback**
3. You need **checkpointing** (save points in workflows)
4. You want to preserve **encapsulation** while saving state
5. State changes are frequent but recoverable

Avoid Memento when:

1. State is huge (memory-intensive)
2. Undo isn't needed
3. State is simple and easily reconstructible
4. Encapsulation isn't a concern

11. Real-World Use Cases

Use Case	How Memento Applies
Database Transactions	Commit/Rollback on failure (our example)
Git Version Control	Every commit is a Memento; checkout = restore
Text Editors (Undo/Redo)	Ctrl+Z restores previous state
Photoshop History Panel	Snapshots of editing steps
Game Save Points	Save game state; load to restore
Workflow Engines	Checkpoint and rollback on error
VCS (Git, SVN)	Commit history as mementos
Banking Systems	Transaction atomicity
Form Auto-Save	Preserve form state on failure

12. Key Takeaways

1. **Memento = Snapshot of object state** without breaking encapsulation
2. **Three roles:** Originator (creates state), Memento (holds snapshot), Caretaker (manages mementos)
3. **Key methods:** `createMemento()` , `restoreFromMemento()` , `getState()` , `setState()`
4. **Real world:** DB transactions, Git, undo/redo, game save
5. **Trade-off:** Memory cost vs. safety/undo capability
6. **Encapsulation** — Caretaker treats Memento as a black box
7. **Two designs:** Simple (single backup) vs. History (stack of snapshots)

13. Interview Tips

When asked about Memento pattern:

- 1. **State the problem:** "Need to save/restore an object's state without breaking encapsulation"
- 2. **Name the three roles:** Originator, Memento, Caretaker — explain each
- 3. **Show the flow:** `createMemento()` → `store` → `restoreFromMemento()`
- 4. **Give a real example:** Database commit/rollback or Ctrl+Z
- 5. **Discuss encapsulation:** Why Caretaker never inspects Memento
- 6. **Mention trade-offs:** Memory cost, need to prune history
- 7. **Compare with Command pattern:** Command does something; Memento restores something

Key phrase: "Memento captures and externalizes an object's internal state so it can be restored later, without violating encapsulation."

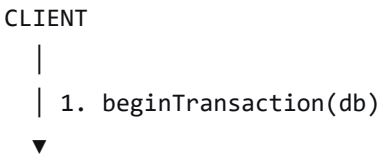
14. Comparison with Related Patterns

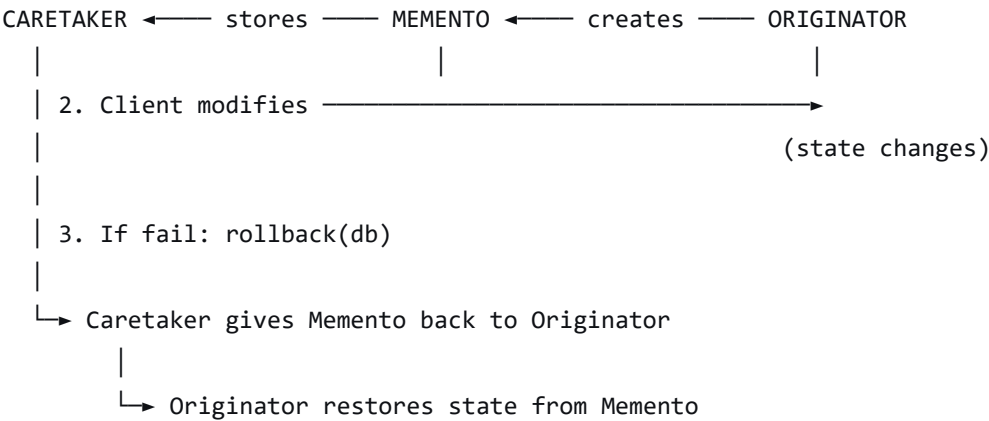
Pattern	Difference from Memento
Command	Encapsulates a request; Memento encapsulates state
Prototype	Creates new objects by cloning; Memento restores an existing object
State	Changes behavior based on state; Memento saves/restores state
Iterator	Traverses elements; Memento captures a snapshot

Memento + Command: Often used together for undo/redo systems. Command encapsulates the operation; Memento captures state before operation.

15. Summary Diagram

text





Simple rule: Originator creates Mementos, Caretaker stores them, Originator restores from them. Caretaker never peeks inside.

 हां जी तो करें स्टार्ट। वेलकम बै...
TXT 61.71KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Anti-Patterns & Null Object Pattern - Complete Notes

Part 1: Anti-Patterns

1. Introduction to Anti-Patterns

An **anti-pattern** is a common response to a recurring problem that is **ineffective and counterproductive**. It's the **opposite of a design pattern** — instead of solving a problem, it creates new problems.

Context: Design Patterns vs Anti-Patterns

Aspect	Design Pattern	Anti-Pattern
Purpose	Solve recurring problems	Describe common mistakes
Outcome	Clean, maintainable code	Messy, tightly coupled code
Example	Strategy, Observer, Visitor	God Object, Spaghetti Code

Aspect	Design Pattern	Anti-Pattern
Awareness	Applied intentionally	Usually accidental

Why Learn Anti-Patterns?

1. **Recognition** — Learn to identify when you're falling into a trap
2. **Prevention** — Avoid common mistakes before they happen
3. **Refactoring** — Know what to fix in existing code
4. **Communication** — Common vocabulary for code problems

2. The Nine Common Anti-Patterns

2.1 God Object

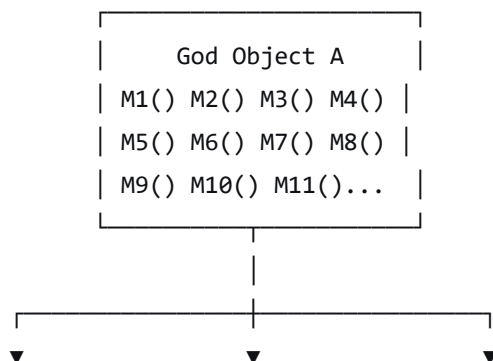
Definition: A class that knows too much or does too much — it becomes a "central hub" for the entire application.

Symptoms:

- One class with many methods handling unrelated responsibilities
- Most operations funnel through it
- Everything is tightly coupled to it
- Breaks **Single Responsibility Principle (SRP)**

Visual:

text



Class B	Class C	Class D
(depends)	(depends)	(depends)

Why It's Bad:

- **Tight Coupling:** Every class depends on the God Object
- **Fragility:** A bug in the God Object breaks the whole application
- **Untestable:** Huge class with hundreds of methods is hard to unit test
- **Violates SRP:** Many reasons for the class to change

Is an Orchestration Class a God Object?

This is a common question! The answer depends:

Scenario	Is it a God Object?
Pure delegation — class just routes calls to other classes	✗ No — it's a Façade
Delegation + business logic — class performs operations then delegates	✓ Yes — it's a God Object
Single entry point — the client has one reference but class is thin	✗ No — it's a Façade

Real-world Example (REST Controllers):

Spring Boot controllers expose APIs like `/v1/add-user` , `/v1/remove-user` . These are **not** God Objects because they merely **delegate** to service layer.

Solution:

- **Distribute responsibilities** across multiple classes
- Use **Façade pattern** — pure delegation with no logic
- Extract cohesive subsets into separate classes

Code Example (Bad vs Good):

Bad (God Object):

```
java
```

```
class ApplicationManager {  
    public void createUser() { /* DB + validation + email */ }
```

```

public void deleteUser() { /* DB + logging + notification */ }
public void sendEmail() { /* SMTP + template + retry */ }
public void processPayment() { /* gateway + fraud + DB */ }
public void generateReport() { /* query + format + cache */ }
// 100+ more methods
}

```

Good (Distributed):

java

```

class UserService { /* user-related only */ }
class EmailService { /* email-related only */ }
class PaymentService { /* payment-related only */ }
class ReportService { /* report-related only */ }

```

Python Example:

python

```

# Bad
class GodManager:
    def create_user(self): ...
    def delete_user(self): ...
    def send_email(self): ...
    def process_payment(self): ...
    def generate_report(self): ...
    # hundreds more...

```

Good

```

class UserService:
    def create(self): ...
    def delete(self): ...

class EmailService:
    def send(self): ...

class PaymentService:
    def process(self): ...

```

2.2 Spaghetti Code

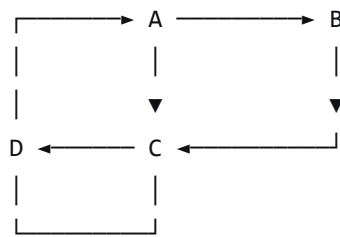
Definition: Code with **no clear structure**, no entry/exit points, tangled dependencies, and cyclic references.

Symptoms:

- Multiple classes referencing each other cyclically
- No clear flow of control
- Difficult to trace execution
- Impossible to understand without extensive study

Visual:

text



(cyclic, tangled, no entry point)

Why It's Bad:

- **Highly Tightly Coupled** — changes propagate unpredictably
- **Error-Prone** — hard to see the impact of changes
- **Unmaintainable** — new developers can't understand it
- **Untestable** — cannot isolate components

Real-world Analogy: Literal spaghetti — tangled noodles with no start or end.**Solution:**

- **Clear layering** (presentation, business, data)
- **Dependency Inversion** — depend on abstractions
- **No cyclic dependencies** — use events or mediators
- **Single entry point** (Façade)
- **Consistent naming and structure**

Code Example:**Bad (Spaghetti):**

java

```

class A {
    B b;
    void doA() { b.doB(); }
}
class B {
    C c;
    A a; // Circular!
    void doB() { c.doC(); a.doA(); }
}
class C {
    A a;
    void doC() { a.doA(); }
}

```

Good (Layered):

java

```

class A { B b; void doA() { b.doB(); } } // A → B
class B { C c; void doB() { c.doC(); } } // B → C
class C { void doC() { /* pure logic */ } } // C has no back-refs

```

Python:

python

Bad – tangled

```

class A:
    def __init__(self, b): self.b = b
    def do_a(self): self.b.do_b()

class B:
    def __init__(self, a, c):
        self.a = a # circular
        self.c = c
    def do_b(self): self.c.do_c(); self.a.do_a()

```

Good – clean layered

```

class A:
    def __init__(self, b): self.b = b
    def do_a(self): self.b.do_b()

class B:
    def __init__(self, c): self.c = c
    def do_b(self): self.c.do_c()

```

```
class C:  
    def do_c(self): ...
```

2.3 Hard Coding

Definition: Embedding fixed values (strings, numbers, paths) directly in code instead of using parameters, constants, or configuration.

Symptoms:

- Strings like "hello" , "localhost:8080" written directly in methods
- Magic numbers (if (x > 42) without explanation)
- Values that could change are buried in code

Why It's Bad:

- **Inflexible** — changing the value requires code changes and redeployment
- **Not Environment-Aware** — dev/prod values get mixed
- **Magic Numbers** — reduces readability
- **Duplication** — same hard-coded value repeated everywhere

Example (Bad):

```
java  
  
public class UserService {  
    public void sendNotification() {  
        String server = "https://prod.api.example.com"; // HARDCODED!  
        send(server, "welcome@example.com"); // HARDCODED!  
    }  
}
```

Example (Good):

```
java  
  
public class UserService {  
    private final String serverUrl;  
    private final String defaultEmail;  
  
    public UserService(String serverUrl, String defaultEmail) {  
        this.serverUrl = serverUrl;  
    }  
}
```

```

        this.defaultEmail = defaultEmail;
    }

    public void sendNotification() {
        send(serverUrl, defaultEmail);
    }
}

```

Python (Using Config):

```

python

import os

# Bad
def send_notification():
    server = "https://prod.api.example.com"
    ...

# Good – use environment/config
class UserService:
    def __init__(self):
        self.server_url = os.getenv("SERVER_URL")
        self.default_email = os.getenv("DEFAULT_EMAIL")

    def send_notification(self):
        ...

```

When It's OK to Hardcode:

- Truly never-changing values ($\pi = 3.14159$)
- Documented constants (e.g., `MAX_RETRIES = 3`)
- Compile-time constants
- Test fixtures where the intent is clarity

2.4 Gold Plating / Over-Engineering

Definition: Adding features, abstractions, or "future-proofing" that is **not required** and may never be needed.

Symptoms:

- Applying design patterns to every piece of code

- Creating abstractions "just in case"
- Handling scenarios that will never happen
- Multiple layers of indirection for simple operations

Why It's Bad:

- **Complexity** — harder to understand simple things
- **Over-abstraction** — premature interfaces with one implementation
- **Dead Code** — unused strategies, visitors, factories
- **Slower Development** — time spent on unused features
- **Risk of Bugs** — more code = more bug surface

Example (Over-engineered):

```
java

// Trying to be "extensible" for a trivial operation
interface GreetingStrategy {
    String greet();
}

class HelloGreeting implements GreetingStrategy {
    public String greet() { return "Hello"; }
}

class GreetingStrategyFactory {
    public static GreetingStrategy create(String type) {
        if (type.equals("hello")) return new HelloGreeting();
        // ... support for 20 Languages "just in case"
    }
}

// Usage: overkill for printing "Hello"
```

Reasonable Version:

```
java

public class Greeter {
    public String greet() { return "Hello"; }
}
```

Guiding Principle:

"You Aren't Gonna Need It" (YAGNI) — Don't add features you don't currently need.

When to Consider Extensibility:

- You have **concrete near-term requirements**
- You already have **two+ real use cases**
- The abstraction is **cheap and simple**

2.5 DRY Violation (Don't Repeat Yourself)

Definition: Duplicating logic across multiple places in the codebase.

Symptoms:

- Copy-pasting code between methods/classes
- Same validation logic in multiple places
- Similar business rules implemented twice

Why It's Bad:

- **Maintenance Nightmare** — change one place, must change all copies
- **Inconsistency** — copies drift apart over time
- **Bug Multiplication** — a bug fix in one copy misses others

Example (Bad):

java

```
class OrderService {
    public void processOrder(Order o) {
        // Validation duplicated
        if (o.getAmount() <= 0) throw new IllegalArgumentException();
        if (o.getCustomer() == null) throw new IllegalArgumentException();
        // ... process
    }

    public void refundOrder(Order o) {
        // SAME validation duplicated!
        if (o.getAmount() <= 0) throw new IllegalArgumentException();
        if (o.getCustomer() == null) throw new IllegalArgumentException();
        // ... refund
    }
}
```

```
    }
}
```

Example (Good):

java

```
class OrderValidator {
    public static void validate(Order o) {
        if (o.getAmount() <= 0) throw new IllegalArgumentException();
        if (o.getCustomer() == null) throw new IllegalArgumentException();
    }
}

class OrderService {
    public void processOrder(Order o) {
        OrderValidator.validate(o);
        // ... process
    }

    public void refundOrder(Order o) {
        OrderValidator.validate(o);
        // ... refund
    }
}
```

Python:

python

```
# Bad
def process_order(order):
    if order['amount'] <= 0: raise ValueError()
    if not order.get('customer'): raise ValueError()
    # ...

def refund_order(order):
    if order['amount'] <= 0: raise ValueError()
    if not order.get('customer'): raise ValueError()
    # ...

# Good
def validate_order(order):
    if order['amount'] <= 0: raise ValueError()
    if not order.get('customer'): raise ValueError()

def process_order(order):
```

```

    validate_order(order)
    # ...

def refund_order(order):
    validate_order(order)
    # ...

```

Rule of Three: It's OK to duplicate once. On the third occurrence, extract.

2.6 Constructor Overloading / Constructor Telescoping

Definition: Creating many overloaded constructors to handle different combinations of parameters.

Symptoms:

- Multiple constructors with different parameter counts
- Client must pick the "right" constructor
- Adding a parameter forces changes to all constructors

Why It's Bad:

- **Cognitive Load** — client confused about which constructor to use
- **Rigidity** — adding a field breaks API
- **Parameter Order Confusion** — `new User("John", 30, "NYC")` VS `new User("NYC", "John", 30)` — easy to mix up
- **Breaks Open/Closed Principle** — new field = new constructor

Example (Bad):

```

java

class User {
    public User() { }
    public User(String name) { }
    public User(String name, int age) { }
    public User(String name, int age, String city) { }
    public User(String name, int age, String city, String email) { }
    public User(String name, int age, String city, String email, String phone) { }
}

```

Solution: Builder Design Pattern

java

```
class User {
    private final String name;
    private final int age;
    private final String city;
    private final String email;

    private User(Builder b) {
        this.name = b.name;
        this.age = b.age;
        this.city = b.city;
        this.email = b.email;
    }

    public static class Builder {
        private String name;
        private int age;
        private String city;
        private String email;

        public Builder name(String name) { this.name = name; return this; }
        public Builder age(int age) { this.age = age; return this; }
        public Builder city(String city) { this.city = city; return this; }
        public Builder email(String email) { this.email = email; return this; }

        public User build() { return new User(this); }
    }
}

// Usage – clean and flexible
User u = new User.Builder()
    .name("Aditya")
    .age(25)
    .city("Mumbai")
    .build();
```

Python:

python

```
class User:
    def __init__(self, name=None, age=None, city=None, email=None):
        self.name = name
        self.age = age
        self.city = city
```

```

        self.email = email

# Or Pythonic builder
class UserBuilder:
    def __init__(self):
        self._data = {}

    def name(self, name): self._data['name'] = name; return self
    def age(self, age): self._data['age'] = age; return self
    def city(self, city): self._data['city'] = city; return self
    def email(self, email): self._data['email'] = email; return self

    def build(self): return User(**self._data)

# Usage
u = UserBuilder().name("Aditya").age(25).build()

```

2.7 Overuse of Getters and Setters

Definition: Generating getters and setters for every field automatically without considering if they should be exposed.

Symptoms:

- Every private field has public getter AND setter
- No validation in setters
- Object internals exposed indiscriminately
- Auto-generated boilerplate in IDE

Why It's Bad:

- **Breaks Encapsulation** — fields marked private for a reason
- **No Invariants** — object can be set to inconsistent state
- **Silent State Mutations** — no control over how state changes
- **Ignores Intent** — why did you make it private in the first place?

Example (Bad):

```

java

class BankAccount {
    private double balance; // Should NOT have public setter!

```

```

    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; } // Dangerous!
}

// Somewhere in code:
account.setBalance(-1000000); // Negative balance - invalid!

```

Example (Good):

java

```

class BankAccount {
    private double balance;

    public double getBalance() { return balance; }

    // Controlled mutations only
    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException();
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0 || amount > balance) throw new IllegalArgumentException();
        balance -= amount;
    }
}

```

Python:

python

```

class BankAccount:
    def __init__(self):
        self._balance = 0

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Invalid amount")
        self._balance += amount

    def withdraw(self, amount):

```

```
if amount <= 0 or amount > self._balance:  
    raise ValueError("Invalid amount")  
self._balance -= amount
```

Rule:

- **Getter** — only if you want to expose reading
- **Setter** — only if you want uncontrolled writing
- **Business methods** — preferred over raw setters (e.g., `deposit()` , `withdraw()`)

2.8 Premature Optimization

Definition: Optimizing code before you know it's a performance bottleneck, at the cost of readability and simplicity.

Symptoms:

- Complex code justified by "performance"
- Caching before measuring
- Micro-optimizations (`i++` vs `++i` obsessive concerns)
- Bitwise tricks where readability matters

Why It's Bad:

- **Complexity** — optimized code is usually harder to read
- **Wrong Focus** — you might optimize something irrelevant
- **Wasted Effort** — most code doesn't need optimization
- **Risk of Bugs** — complex optimizations introduce subtle bugs

Famous Quote:

"Premature optimization is the root of all evil." — Donald Knuth

Corollary:

"Make it work, then make it fast." — Kent Beck

Example (Premature Optimization):

java

```
// Bitwise trick to check even – but "i % 2 == 0" is clearer and JIT optimizes it anyway  
if ((i & 1) == 0) { /* even */ }
```

Correct Approach:

java

```
if (i % 2 == 0) { /* even – clear, and JIT compiler optimizes */ }
```

DS&A Analogy:

- Start with **brute force** ($O(n^2)$ is OK)
- Once working, **optimize** to $O(n \log n)$ if needed
- Measure before optimizing (profiler, benchmark)

Guideline:

1. **Make it work** — correctness first
2. **Make it right** — clean code, good structure
3. **Make it fast** — only if measurements say so

2.9 Inheritance Overuse

Definition: Using inheritance for code reuse when composition would be better.

Symptoms:

- Deep inheritance hierarchies ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow \dots$)
- Multiple inheritance used for behavior
- Subclasses overriding most parent behavior
- "is-a" relationship claimed where it's actually "has-a"

Why It's Bad:

- **Fragile Base Class Problem** — changing parent breaks children
- **Tight Coupling** — subclasses depend on implementation details
- **Diamond Problem** — multiple inheritance ambiguity
- **Liskov Substitution Violations** — subclass may not be a true subtype

Example (Bad — Inheritance):

java

```
class Robot {
    public void fly() { /* generic flying */ }
    public void walk() { /* generic walking */ }
}

class FlyingRobot extends Robot {
    // Inherits walk() but flying robot shouldn't walk
}
```

Better — Composition:

java

```
interface FlyBehavior { void fly(); }
interface WalkBehavior { void walk(); }

class FlyWithWings implements FlyBehavior { /* ... */ }
class NoFly implements FlyBehavior { /* ... */ }

class Robot {
    private FlyBehavior flyBehavior;
    private WalkBehavior walkBehavior;

    // Constructor injection
    // Delegation methods
}
```

Preferred Approach:

- **Inherit** only from **abstract classes** for interface contracts
- **Compose** for behavior (has-a relationship)
- **Use Strategy, Bridge, Visitor** to avoid inheritance hierarchies

Python Example:

python

```
# Bad — inheritance for behavior
class Robot:
    def fly(self): pass
    def walk(self): pass
```

```
class FlyingRobot(Robot):  
    def walk(self): # Override to disable - smell!  
        pass  
  
# Good - composition  
class Robot:  
    def __init__(self, fly_behavior, walk_behavior):  
        self._fly = fly_behavior  
        self._walk = walk_behavior  
  
    def fly(self): self._fly.do()  
    def walk(self): self._walk.do()
```

3. Summary Table of Anti-Patterns

#	Anti-Pattern	Core Problem	Primary Fix
1	God Object	Too many responsibilities	Split, use Façade for pu delegation
2	Spaghetti Code	No clear structure, cyclic deps	Layering, Dependency Inversion
3	Hard Coding	Fixed values in code	Config, constants, parameters
4	Gold Plating	Over-engineering for unlikely cases	YAGNI, only abstract when needed
5	DRY Violation	Copy-pasted logic	Extract to shared method/class
6	Constructor Overloading	Telescoping constructors	Builder pattern
7	Overuse of Getters/Setters	Broken encapsulation	Business methods, controlled mutation
8	Premature Optimization	Complexity without evidence	Make it work → right → fast

#	Anti-Pattern	Core Problem	Primary Fix
9	Inheritance Overuse	Fragile base class, deep hierarchies	Composition, Strategy, Bridge

Part 2: Null Object Pattern

1. Introduction

The **Null Object Pattern** is a behavioral design pattern that:

- Provides an **object as a surrogate** for a null reference
- The Null Object **does nothing** (no-op) when methods are called
- **Eliminates null checks** throughout the codebase
- Prevents `NullPointerException` / `AttributeError`

Standard Definition

"Avoid null checks by providing a neutral, no-op object that behaves like a valid object but does nothing."

2. The Problem It Solves

Without Null Object Pattern

Every time you use an object that might be null, you need checks:

```
java
if (obj != null) {
    obj.doSomething();
} else {
    // do nothing
}
```

Problems:

1. **Easy to forget** — programmer might skip the check
2. **Verbose** — null checks clutter code
3. **Distributed** — same check must be applied everywhere
4. **Runtime Errors** — `NullPointerException` when forgotten
5. **Violates "Replace Conditionals with Polymorphism"** — OOP prefers polymorphism over `if/else`

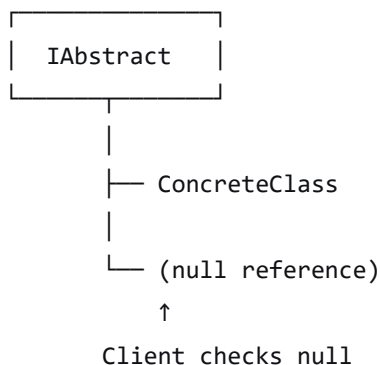
3. How It Works

Add a **Null Object** subclass that:

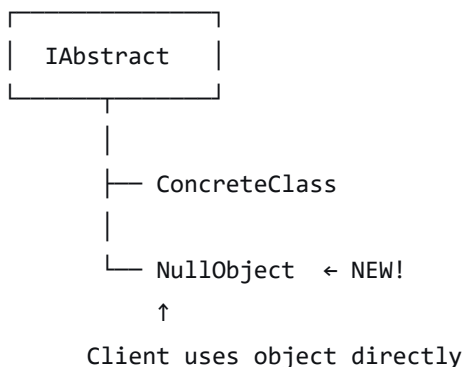
- Inherits from the same interface/abstract class
- Implements all methods as **no-ops** (empty body) or **default values**

text

Before:

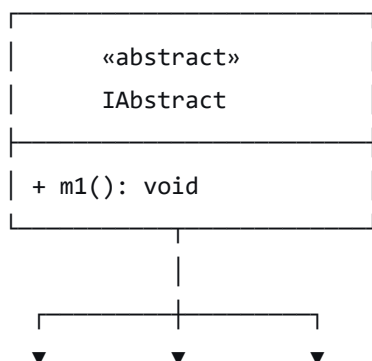


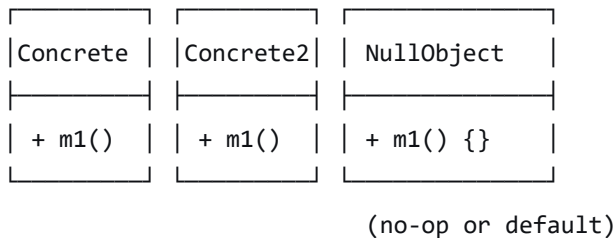
After:



4. UML Class Diagram

text





5. Java Implementation

5.1 Abstract Class

java

```
public abstract class AbstractRobot {
    public abstract void fly();
    public abstract void talk();
    public abstract void walk();
}
```

5.2 Concrete Classes

java

```
public class RealRobot extends AbstractRobot {
    private String name;

    public RealRobot(String name) {
        this.name = name;
    }

    @Override
    public void fly() { System.out.println(name + " is flying"); }

    @Override
    public void talk() { System.out.println(name + " is talking"); }

    @Override
    public void walk() { System.out.println(name + " is walking"); }
}
```

5.3 Null Object

java

```
public class NullRobot extends AbstractRobot {  
    @Override  
    public void fly() { /* no-op – do nothing */ }  
  
    @Override  
    public void talk() { /* no-op */ }  
  
    @Override  
    public void walk() { /* no-op */ }  
}
```

5.4 Client

java

```
public class Client {  
    private AbstractRobot robot;  
  
    public Client(AbstractRobot robot) {  
        // Never null! If null is passed, substitute NullRobot  
        this.robot = (robot != null) ? robot : new NullRobot();  
    }  
  
    public void execute() {  
        // No null check needed!  
        robot.fly();  
        robot.talk();  
        robot.walk();  
    }  
  
    public static void main(String[] args) {  
        Client c1 = new Client(new RealRobot("R2D2"));  
        c1.execute(); // All methods execute  
  
        Client c2 = new Client(null); // No robot  
        c2.execute(); // Silent no-ops – no exception!  
    }  
}
```

Expected Output

text

R2D2 is flying
R2D2 is talking
R2D2 is walking
(no output from Client 2 – silent no-ops)

6. Python Implementation

python

```
from abc import ABC, abstractmethod
```

```
class AbstractRobot(ABC):  
    @abstractmethod  
    def fly(self) -> None: ...  
  
    @abstractmethod  
    def talk(self) -> None: ...  
  
    @abstractmethod  
    def walk(self) -> None: ...
```

```
class RealRobot(AbstractRobot):  
    def __init__(self, name: str):  
        self.name = name  
  
    def fly(self) -> None:  
        print(f"{self.name} is flying")  
  
    def talk(self) -> None:  
        print(f"{self.name} is talking")  
  
    def walk(self) -> None:  
        print(f"{self.name} is walking")
```

```
class NullRobot(AbstractRobot):  
    """Null Object – silently does nothing"""  
  
    def fly(self) -> None:  
        pass # no-op  
  
    def talk(self) -> None:  
        pass
```

```

def walk(self) -> None:
    pass

class Client:
    def __init__(self, robot: AbstractRobot = None):
        # Substitute null with NullRobot
        self.robot = robot if robot is not None else NullRobot()

    def execute(self) -> None:
        # No null checks!
        self.robot.fly()
        self.robot.talk()
        self.robot.walk()

if __name__ == "__main__":
    c1 = Client(RealRobot("R2D2"))
    c1.execute()

    c2 = Client(None) # No robot
    c2.execute()      # Silent no-ops

```

Pythonic Alternative:

python

```

class NullRobot(AbstractRobot):
    def __getattr__(self, name):
        # Any method returns a no-op
        return lambda *args, **kwargs: None

```

7. Real-World Applications (From This Series)

7.1 Strategy Pattern — "No Fly Behavior"

In the Robot example from the Strategy pattern:

java

```

interface IFlyStrategy {
    void fly();
}

```

```

class FlyWithWings implements IFlyStrategy { /* real fly */ }
class FlyWithJet implements IFlyStrategy { /* jet fly */ }

// Null Object
class NoFly implements IFlyStrategy {
    public void fly() {
        // Intentionally does nothing
    }
}

```

Usage: When a robot can't fly, use `NoFly` instead of `null` . Client code avoids:

```

java

if (robot.getFlyStrategy() != null) {
    robot.getFlyStrategy().fly();
}

```

7.2 Command Pattern — "No Command"

In a Remote Control scenario:

```

java

interface ICommand {
    void execute();
}

class LightOnCommand implements ICommand { /* turn on */ }
class LightOffCommand implements ICommand { /* turn off */ }

// Null Object
class NoCommand implements ICommand {
    public void execute() {
        // Nothing to do
    }
}

```

Usage: Buttons with no command get `NoCommand` by default. Press them → nothing happens, but no `NullPointerException` .

8. Advantages & Disadvantages

Advantages

- ✓ **No null checks** — cleaner client code
- ✓ **No NullPointerException** — safer runtime
- ✓ **Polymorphism** — replaces conditionals with OOP
- ✓ **Liskov Substitution** — null object is a valid substitute
- ✓ **Predictable defaults** — for return values, empty collections

Disadvantages

- ✗ **Silent failures** — bugs may be hidden (did nothing vs. forgot to do)
- ✗ **Extra class** — one more class to maintain
- ✗ **Behavioral assumption** — caller may expect an effect
- ✗ **Not suitable for all cases** — errors may need reporting

When to Use

Use Null Object when:

1. You have **frequent null checks** for a type
2. **Doing nothing** is a valid behavior
3. You want **polymorphic clients** without conditionals
4. **Default/empty behavior** is acceptable

Don't use when:

1. Null truly signals **an error condition** that must be reported
2. Caller **needs to know** nothing happened
3. Method calls have **side effects** that must occur

9. Comparison with Related Patterns

Pattern	Relation to Null Object
Strategy	Null Object is a Strategy implementation (e.g., NoFly)
Command	Null Object is a Command implementation (e.g., NoCommand)

Pattern	Relation to Null Object
Special Case	Similar — handles edge-case value distinctly
State	Null Object could be a State (e.g., IdleState)
Facade	Façade for simple delegation; Null Object for no-op

10. Key Takeaways

Anti-Patterns (Part 1)

1. **God Object** — one class doing too much
2. **Spaghetti Code** — tangled, unstructured code
3. **Hard Coding** — fixed values in code
4. **Gold Plating** — over-engineering (YAGNI!)
5. **DRY Violation** — copy-pasted logic
6. **Constructor Overloading** — telescoping constructors → use Builder
7. **Overuse of Getters/Setters** — break encapsulation
8. **Premature Optimization** — measure first!
9. **Inheritance Overuse** — prefer composition

Null Object Pattern (Part 2)

1. Replaces `null` with a **no-op object**
2. **Eliminates null checks** in client code
3. **Prevents NullPointerException**
4. Uses **polymorphism** instead of `if (obj != null)`
5. Applied in **Strategy** (NoFly), **Command** (NoCommand)
6. **Liskov Substitution Principle** compliant

11. Interview Tips

For Anti-Patterns

1. **Mention by name** — interviewers love specific vocabulary
2. **Give examples** — "God Object is like a 5000-line class doing everything"
3. **Show awareness** — "I try to avoid these when designing"
4. **Highlight the fix** — "We'd refactor into separate services"
5. **Balance** — "Design patterns aren't rigid rules — sometimes trade-offs are needed"

For Null Object Pattern

1. **Purpose** — "Replaces null checks with a no-op object"
2. **Real examples** — "Strategy's NoFly, Command's NoCommand"
3. **Benefit** — "Cleaner code, no NPEs"
4. **Caution** — "Silent failures — must be used carefully"
5. **Compare** — "It's essentially a Strategy pattern for handling null"

12. Complete Summary

Anti-Patterns are **common mistakes** — the flip side of design patterns:

- They emerge from **well-intentioned shortcuts**
- They cause **tight coupling, complexity, and bugs**
- Recognizing them lets you **write better code proactively**

Null Object Pattern is a **small but powerful design pattern**:

- Substitutes `null` with a **no-op object**
- Uses **polymorphism** to eliminate conditionals
- Applied where **doing nothing is a valid behavior**
- Frequently appears in **Strategy** and **Command** patterns

Together, these concepts complete your LLD toolkit:

- **Design patterns** — positive solutions
- **Anti-patterns** — negative traps to avoid
- **Null Object** — elegant handling of "no object" scenarios

